MySQL 面试题



大家好，我是小林。

MySQL 是后端开发面试里被问烂了的一个话题，但被问烂了不代表好准备，反而越是常见的技术，面试官越能问出深度来。很多人平时项目里天天用 MySQL，自以为没问题，结果面试被问到索引的底层原理、MVCC 的实现机制、或者事务隔离级别的具体区别时，就说不清楚了。

这篇文章把 MySQL 面试中从基础到进阶的高频考点都整理进来了，包括 SQL 基础、存储引擎、索引原理、事务、锁机制、日志系统，到最后的主从复制和分库分表，基本上是一个比较完整的 MySQL 知识体系。内容偏向原理，不只是告诉你"怎么用"，更多是讲清楚"为什么"。

有几块内容建议重点花时间：

- **索引**：B+ 树的结构、聚簇索引和非聚簇索引的区别、联合索引的最左匹配原则、索引失效的场景，这块是 MySQL 面试里出现频率最高的。
- **事务和 MVCC**：四种隔离级别的区别、MVCC 是怎么用 Read View 来实现可重复读的，这是理解并发问题的核心。
- **锁**：行锁、间隙锁、临键锁各自锁的范围，以及什么情况下会升级成表锁，这块很多人答不清楚。
- **日志**：redo log、undo log、binlog 三个日志各自的作用和区别，两阶段提交是怎么保证数据一致性的。

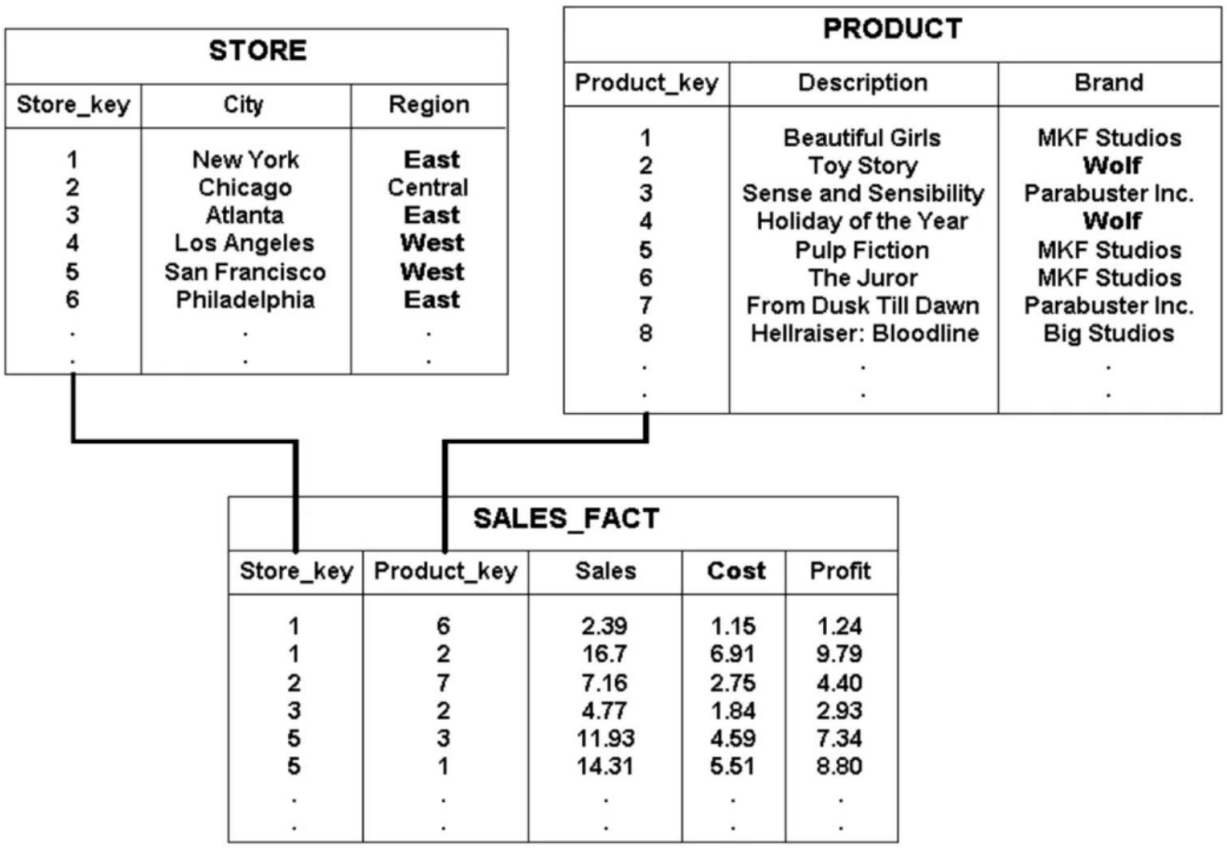
如果你是第一次系统准备 MySQL 面试，建议先搞清楚存储引擎和索引，再去看事务和锁，最后看日志和架构部分。这样知识之间的关联会更清晰，不容易学了后面忘了前面。

SQL 基础

NOSQL 和 SQL 的区别?

SQL 数据库，指关系型数据库 - 主要代表：SQL Server，Oracle，MySQL(开源)，PostgreSQL(开源)。

关系型数据库存储结构化数据。这些数据逻辑上以行列二维表的形式存在，每一列代表数据的一种属性，每一行代表一个数据实体。



STORE

Store_key	City	Region
1	New York	East
2	Chicago	Central
3	Atlanta	East
4	Los Angeles	West
5	San Francisco	West
6	Philadelphia	East

PRODUCT

Product_key	Description	Brand
1	Beautiful Girls	MKF Studios
2	Toy Story	Wolf
3	Sense and Sensibility	Parabuster Inc.
4	Holiday of the Year	Wolf
5	Pulp Fiction	MKF Studios
6	The Juror	MKF Studios
7	From Dusk Till Dawn	Parabuster Inc.
8	Hellraiser: Bloodline	Big Studios

SALES_FACT

Store_key	Product_key	Sales	Cost	Profit
1	6	2.39	1.15	1.24
1	2	16.7	6.91	9.79
2	7	7.16	2.75	4.40
3	2	4.77	1.84	2.93
5	3	11.93	4.59	7.34
5	1	14.31	5.51	8.80

NoSQL 指非关系型数据库 ，主要代表：MongoDB，Redis。NoSQL 数据库逻辑上提供了不同于二维表的存储方式，存储方式可以是 JSON 文档、哈希表或者其他方式。



选择 SQL vs NoSQL，考虑以下因素。

> ACID vs BASE

关系型数据库支持 ACID 即原子性，一致性，隔离性和持续性。相对而言，NoSQL 采用更宽松的模型 BASE ， 即基本可用，软状态和最终一致性。

从实用的角度出发，我们需要考虑对于面对的应用场景，ACID 是否是必须的。比如银行应用就必须保证 ACID，否则一笔钱可能被使用两次；又比如社交软件不必保证 ACID，因为一条状态的更新对于所有用户读取先后时间有数秒不同并不影响使用。

对于需要保证 ACID 的应用，我们可以优先考虑 SQL。反之则可以优先考虑 NoSQL。

> 扩展性对比

NoSQL 数据之间无关系，这样就非常容易扩展，也无形之间，在架构的层面上带来了可扩展的能力。比如 redis 自带主从复制模式、哨兵模式、切片集群模式。

相反关系型数据库的数据之间存在关联性，水平扩展较难 ，需要解决跨服务器 JOIN，分布式事务等问题。

数据库三大范式是什么?

第一范式（1NF）：要求数据库表的每一列都是不可分割的原子数据项。

举例说明：

学号	姓名	性别	家庭信息	学校信息
20150001	李白	男	3口人，北京	硕士，研二
20150002	杜甫	男	2口人，南京	本科，大四
20150003	王维	男	4口人，上海	本科，大三
20150004	白居易	男	3口人，河南	硕士，研一
20150005	刘禹锡	男	4口人，上海	本科，大一
20150006	李清照	女	5口人，西安	硕士，研三
20150007	苏轼	男	2口人，南京	大专，大二
20150008	屈原	男	4口人，吉林	博士，博五
20150009	陶渊明	男	1口人，长沙	博士，博三

在上面的表中，“家庭信息”和“学校信息”列均不满足原子性的要求，故不满足第一范式，调整如下：

学号	姓名	性别	家庭人口	户籍	学历	所在年级
20150001	李白	男	3口人	北京	硕士	研二
20150002	杜甫	男	2口人	南京	本科	大四
20150003	王维	男	4口人	上海	本科	大三
20150004	白居易	男	3口人	河南	硕士	研一
20150005	刘禹锡	男	4口人	上海	本科	大一
20150006	李清照	女	5口人	西安	硕士	研三
20150007	苏轼	男	2口人	南京	大专	大二
20150008	屈原	男	4口人	吉林	博士	博五
20150009	陶渊明	男	1口人	长沙	博士	博三

可见，调整后的每一列都是不可再分的，因此满足第一范式（1NF）；

第二范式（2NF）：在 1NF 的基础上，非码属性必须完全依赖于候选码（在 1NF 基础上消除非主属性对主码的部分函数依赖）

第二范式需要确保数据库表中的每一列都和主键相关，而不能只与主键的某一部分相关（主要针对联合主键而言）。

举例说明：

订单号	产品号	产品数量	产品折扣	产品价格	订单金额	订单时间
2008003	205	100	0.9	8.9	2870	20080103
2008003	206	200	0.8	9.9	2870	20080103
2008005	207	200	0.75	10	2000	20080203
2008006	207	400	0.85	12	4800	20080206
2008007	207	1000	0.88	14	14000	20080209
2008008	210	240	0.95	8	12255	20100423
2008008	211	300	0.75	8	12255	20100423
2008008	212	350	0.8	15.9	12255	20100423

在上图所示的情况中，同一个订单中可能包含不同的产品，因此主键必须是“订单号”和“产品号”联合组成，

但可以发现，产品数量、产品折扣、产品价格与“订单号”和“产品号”都相关，但是订单金额和订单时间仅与“订单号”相关，与“产品号”无关，

这样就不满足第二范式的要求，调整如下，需分成两个表：

订单号	产品号	产品数量	产品折扣	产品价格
2008003	205	100	0.9	8.9
2008003	206	200	0.8	9.9
2008005	207	200	0.75	10
2008006	207	400	0.85	12
2008007	207	1000	0.88	14
2008008	210	240	0.95	8
2008008	211	300	0.75	8
2008008	212	350	0.8	15.9

订单号	订单金额	订单时间
2008003	2870	20080103
2008003	2870	20080103
2008005	2000	20080203
2008006	4800	20080206
2008007	14000	20080209
2008008	12255	20100423
2008008	12255	20100423
2008008	12255	20100423

第三范式（3NF）：在 2NF 基础上，任何非主属性 (opens new window)不依赖于其它非主属性（在 2NF 基础上消除传递依赖）

第三范式需要确保数据表中的每一列数据都和主键直接相关，而不能间接相关。

举例说明：

学号	姓名	性别	家庭人口	班主任姓名	班主任性别	班主任年龄
20150001	李白	男	3口人	陈洁	女	35
20150002	杜甫	男	2口人	陈洁	女	35
20150003	王维	男	4口人	陈洁	女	35
20150004	白居易	男	3口人	李丽	女	32
20150005	刘禹锡	男	4口人	李丽	女	32
20150006	李清照	女	5口人	王安	男	29
20150007	苏轼	男	2口人	南林	男	34
20150008	屈原	男	4口人	南林	男	34
20150009	陶渊明	男	1口人	王安	男	29

上表中，所有属性都完全依赖于学号，所以满足第二范式，但是“班主任性别”和“班主任年龄”直接依赖的是“班主任姓名”，

而不是主键“学号”，所以需做如下调整：

学号	姓名	性别	家庭人口	班主任姓名
20150001	李白	男	3口人	陈洁
20150002	杜甫	男	2口人	陈洁
20150003	王维	男	4口人	陈洁
20150004	白居易	男	3口人	李丽
20150005	刘禹锡	男	4口人	李丽
20150006	李清照	女	5口人	王安
20150007	苏轼	男	2口人	南林
20150008	屈原	男	4口人	南林
20150009	陶渊明	男	1口人	王安

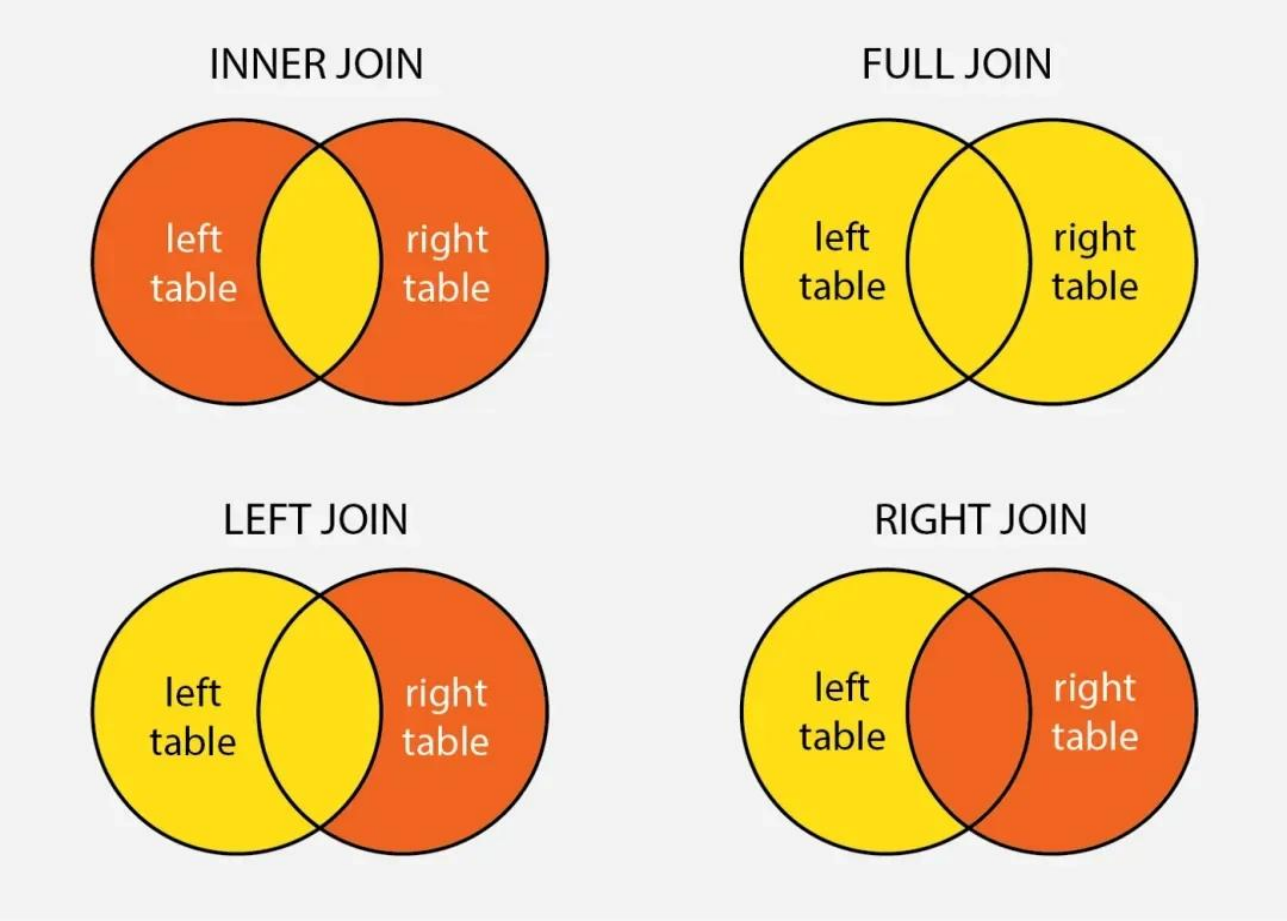
班主任姓名	班主任性别	班主任年龄
陈洁	女	35
陈洁	女	35
陈洁	女	35
李丽	女	32
李丽	女	32
王安	男	29
南林	男	34
南林	男	34
王安	男	29

这样以来，就满足了第三范式的要求。

MySQL 怎么连表查询?

数据库有以下几种联表查询类型：

1. **内连接 (INNER JOIN)**
2. **左外连接 (LEFT JOIN)**
3. **右外连接 (RIGHT JOIN)**
4. **全外连接 (FULL JOIN)**



1. 内连接 (INNER JOIN)

内连接返回两个表中有匹配关系的行。**示例**:

```
SELECT employees.name, departments.name
FROM employees
INNER JOIN departments
ON employees.department_id = departments.id;
```

这个查询返回每个员工及其所在的部门名称。

2. 左外连接 (LEFT JOIN)

左外连接返回左表中的所有行，即使在右表中没有匹配的行。未匹配的右表列会包含 NULL。**示例**:

```
SELECT employees.name, departments.name
FROM employees
LEFT JOIN departments
ON employees.department_id = departments.id;
```

这个查询返回所有员工及其部门名称，包括那些没有分配部门的员工。

3. 右外连接 (RIGHT JOIN)

右外连接返回右表中的所有行，即使左表中没有匹配的行。未匹配的左表列会包含 NULL。**示例**:

```
SELECT employees.name, departments.name
FROM employees
RIGHT JOIN departments
ON employees.department_id = departments.id;
```

这个查询返回所有部门及其员工，包括那些没有分配员工的部门。

4. 全外连接 (FULL JOIN)

全外连接返回两个表中所有行，包括非匹配行，在 MySQL 中，FULL JOIN 需要使用 UNION 来实现，因为 MySQL 不直接支持 FULL JOIN。**示例**:

```
SELECT employees.name, departments.name
FROM employees
LEFT JOIN departments
ON employees.department_id = departments.id

UNION

SELECT employees.name, departments.name
FROM employees
RIGHT JOIN departments
ON employees.department_id = departments.id;
```

这个查询返回所有员工和所有部门，包括没有匹配行的记录。

MySQL 如何避免重复插入数据?

方式一：使用 UNIQUE 约束

在表的相关列上添加 UNIQUE 约束，确保每个值在该列中唯一。例如：

```
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    email VARCHAR(255) UNIQUE,
    name VARCHAR(255)
```

```
);
```

如果尝试插入重复的 email，MySQL 会返回错误。

方式二：使用 INSERT ... ON DUPLICATE KEY UPDATE

这种语句允许在插入记录时处理重复键的情况。如果插入的记录与现有记录冲突，可以选择更新现有记录：

```
INSERT INTO users (email, name)
VALUES ('example@example.com', 'John Doe')
ON DUPLICATE KEY UPDATE name = VALUES(name);
```

方式三：使用 INSERT IGNORE： 该语句会在插入记录时忽略那些因重复键而导致的插入错误。例如：

```
INSERT IGNORE INTO users (email, name)
VALUES ('example@example.com', 'John Doe');
```

如果 email 已经存在，这条插入语句将被忽略而不会返回错误。

选择哪种方法取决于具体的需求：

- 如果需要保证全局唯一性，使用 UNIQUE 约束是最佳做法。
- 如果需要插入和更新结合可以使用 `ON DUPLICATE KEY UPDATE`。
- 对于快速忽略重复插入，`INSERT IGNORE` 是合适的选择。

CHAR 和 VARCHAR 有什么区别?

- CHAR 是固定长度的字符串类型，定义时需要指定固定长度，存储时会在末尾补足空格。CHAR 适合存储长度固定的数据，如固定长度的代码、状态等，存储空间固定，对于短字符串效率较高。
- VARCHAR 是可变长度的字符串类型，定义时需要指定最大长度，实际存储时根据实际长度占用存储空间。VARCHAR 适合存储长度可变的数据，如用户输入的文本、备注等，节约存储空间。

varchar 后面代表字节还是会字符?

`VARCHAR` 后面括号里的数字代表的是字符数，而不是字节数。

比如 `VARCHAR(10)`，这里的 10 表示该字段最多可以存储 10 个字符。字符的字节长度取决于所使用的字符集。

- 如果字符集是 ASCII 字符集：ASCII 字符集每个字符占用 1 个字节，那么 VARCHAR(10) 最多可以存储 10 个 ASCII 字符，同时占用的存储空间最多为 10 个字节（不考虑额外的长度记录开销）。
- 如果字符集是 UTF - 8 字符集，它的每个字符可能占用 1 到 4 个字节，对于 `VARCHAR(10)` 的字段，它最多可以存储 10 个字符，但占用的字节数会根据字符的不同而变化。

int(1) int(10) 在 mysql 有什么不同?

`INT(1)` 和 `INT(10)` 的区别主要在于 **显示宽度**，而不是存储范围或数据类型本身的大小。以下是核心区别的总结：

- 本质是显示宽度，不改变存储方式：`INT` 的存储固定为 4 字节，所有 `INT`（无论写成 `INT(1)` 还是 `INT(10)`）占用的存储空间 均为 4 字节。括号内的数值（如 `1` 或 `10`）是显示宽度，用于在 特定场景下 控制数值的展示格式。
- 唯一作用场景：`ZEROFILL` 补零显示，当字段设置 `ZEROFILL` 时：数字显示时会用前导零填充至指定宽度。比如，字段类型为 `INT(4) ZEROFILL`，实际存入 `5` → 显示为 `0005`，实际存入 `12345` → 显示仍为 `12345`（宽度超限时不截断）。

举一个例子

```
-- 创建一个包含 INT(1) 和 INT(10) 字段的表，并设置 ZEROFILL 属性
CREATE TABLE test_int (
    num1 INT(1) ZEROFILL,
    num2 INT(10) ZEROFILL
);

-- 插入数据
INSERT INTO test_int (num1, num2) VALUES (1, 1);

-- 查询数据
SELECT * FROM test_int;
```

结果分析：

- `num1` 字段由于设置为 `INT(1) ZEROFILL`，其显示宽度为 1，插入数据 `1` 时会显示为 `1`。

- `num2` 字段设置为 `INT(10) ZEROFILL`，显示宽度为 10，插入数据 `1` 时会在前面填充零，显示为 `0000000001`。

Text 数据类型可以无限大吗?

MySQL 3 种 text 类型的最大长度如下：

- TEXT：65,535 bytes ~64kb
- MEDIUMTEXT：16,777,215 bytes ~16Mb
- LONGTEXT：4,294,967,295 bytes ~4Gb

IP 地址如何在数据库里存储?

IPv4 地址是一个 32 位的二进制数，通常以点分十进制表示法呈现，例如 `192.168.1.1`。

字符串类型的存储方式：直接将 IP 地址作为字符串存储在数据库中，比如可以用 `VARCHAR(15)`来存储。

```
-- 创建一个表，使用 VARCHAR 类型存储 IPv4 地址
CREATE TABLE ip_records (
    id INT AUTO_INCREMENT PRIMARY KEY,
    ip_address VARCHAR(15)
);

-- 插入数据
INSERT INTO ip_records (ip_address) VALUES ('192.168.1.1');
```

- **优点**：直观易懂，方便直接进行数据的插入、查询和显示，不需要进行额外的转换操作。
- **缺点**：占用存储空间较大，字符串比较操作的性能相对较低，不利于进行范围查询。

整数类型的存储方式：将 IPv4 地址转换为 32 位无符号整数进行存储，常用的数据类型有 `INT UNSIGNED`。

```
-- 创建一个表，使用 INT UNSIGNED 类型存储 IPv4 地址
CREATE TABLE ip_records (
    id INT AUTO_INCREMENT PRIMARY KEY,
    ip_address INT UNSIGNED
);
```

```
-- 插入数据，需要先将 IP 地址转换为整数
INSERT INTO ip_records (ip_address) VALUES (INET_ATON('192.168.1.1'));


-- 查询时将整数转换回 IP 地址
SELECT INET_NTOA(ip_address) FROM ip_records;
```

- **优点**：占用存储空间小，整数比较操作的性能较高，便于进行范围查询。
- **缺点**：需要进行额外的转换操作，不够直观，增加了开发的复杂度。

说一下外键约束

外键约束的作用是维护表与表之间的关系，确保数据的完整性和一致性。让我们举一个简单的例子：

假设你有两个表，一个是学生表，另一个是课程表，这两个表之间有一个关系，即一个学生可以选修多门课程，而一门课程也可以被多个学生选修。在这种情况下，我们可以在学生表中定义一个指向课程表的外键，如下所示：

```
CREATE TABLE students (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  course_id INT,
  FOREIGN KEY (course_id) REFERENCES courses(id)
);
```

这里，`students` 表中的 `course_id` 字段是一个外键，它指向 `courses` 表中的 `id` 字段。这个外键约束确保了每个学生所选的课程在 `courses` 表中都存在，从而维护了数据的完整性和一致性。

如果没有定义外键约束，那么就有可能出现学生选了不存在的课程或者删除了一个课程而忘记从学生表中删除选修该课程的学生的情况，这会破坏数据的完整性和一致性。因此，使用外键约束可以帮助我们避免这些问题。

MySQL 的关键字 in 和 exist

在 MySQL 中，`IN` 和 `EXISTS` 都是用来处理子查询的关键词，但它们在功能、性能和使用场景上有各自的特点和区别。

> IN 关键字

`IN` 用于检查左边的表达式是否存在于右边的列表或子查询的结果集中。如果存在，则 `IN` 返回 `TRUE`，否则返回 `FALSE`。

语法结构：

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (value1, value2, ...);
```

或

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (SELECT column_name FROM another_table WHERE condition);
```

例子：

```
SELECT * FROM Customers
WHERE Country IN ('Germany', 'France');
```

> EXISTS 关键字

`EXISTS` 用于判断子查询是否至少能返回一行数据。它不关心子查询返回什么数据，只关心是否有结果。如果子查询有结果，则 `EXISTS` 返回 `TRUE`，否则返回 `FALSE`。

语法结构：

```
SELECT column_name(s)
FROM table_name
WHERE EXISTS (SELECT column_name FROM another_table WHERE condition);
```

例子：

```
SELECT * FROM Customers
WHERE EXISTS (SELECT 1 FROM Orders WHERE Orders.CustomerID = Customers.CustomerID);
```

区别与选择：

- **性能差异**：在很多情况下，`EXISTS` 的性能优于 `IN`，特别是当子查询的表很大时。这是因为 `EXISTS` 一旦找到匹配项就会立即停止查询，而 `IN` 可能会扫描整个子查询结果集。

- **使用场景**：如果子查询结果集较小且不频繁变动，`IN` 可能更直观易懂。而当子查询涉及外部查询的每一行判断，并且子查询的效率较高时，`EXISTS` 更为合适。
- **NULL 值处理**：`IN` 能够正确处理子查询中包含 NULL 值的情况，而 `EXISTS` 不受子查询结果中 NULL 值的影响，因为它关注的是行的存在性，而不是具体值。

mysql 中的一些基本函数，你知道哪些?

> 一、字符串函数

CONCAT(str1, str2, ...)：连接多个字符串，返回一个合并后的字符串。

```
SELECT CONCAT('Hello', ' ', 'World') AS Greeting;
```

LENGTH(str)：返回字符串的长度（字符数）。

```
SELECT LENGTH('Hello') AS StringLength;
```

SUBSTRING(str, pos, len)：从指定位置开始，截取指定长度的子字符串。

```
SELECT SUBSTRING('Hello World', 1, 5) AS SubStr;
```

REPLACE(str, from_str, to_str)：将字符串中的某部分替换为另一个字符串。

```
SELECT REPLACE('Hello World', 'World', 'MySQL') AS ReplacedStr;
```

> 二、数值函数

ABS(num)：返回数字的绝对值。

```
SELECT ABS(-10) AS AbsoluteValue;
```

POWER(num, exponent)：返回指定数字的指定幂次方。

```
SELECT POWER(2, 3) AS PowerValue;
```

> 三、日期和时间函数

NOW()：返回当前日期和时间。

```
SELECT NOW() AS CurrentDateTime;
```

CURDATE()：返回当前日期。

```
SELECT CURDATE() AS CurrentDate;
```

> 四、聚合函数

COUNT(column)：计算指定列中的非 NULL 值的个数。

```
SELECT COUNT(*) AS RowCount FROM my_table;
```

SUM(column)：计算指定列的总和。

```
SELECT SUM(price) AS TotalPrice FROM orders;
```

AVG(column)：计算指定列的平均值。

```
SELECT AVG(price) AS AveragePrice FROM orders;
```

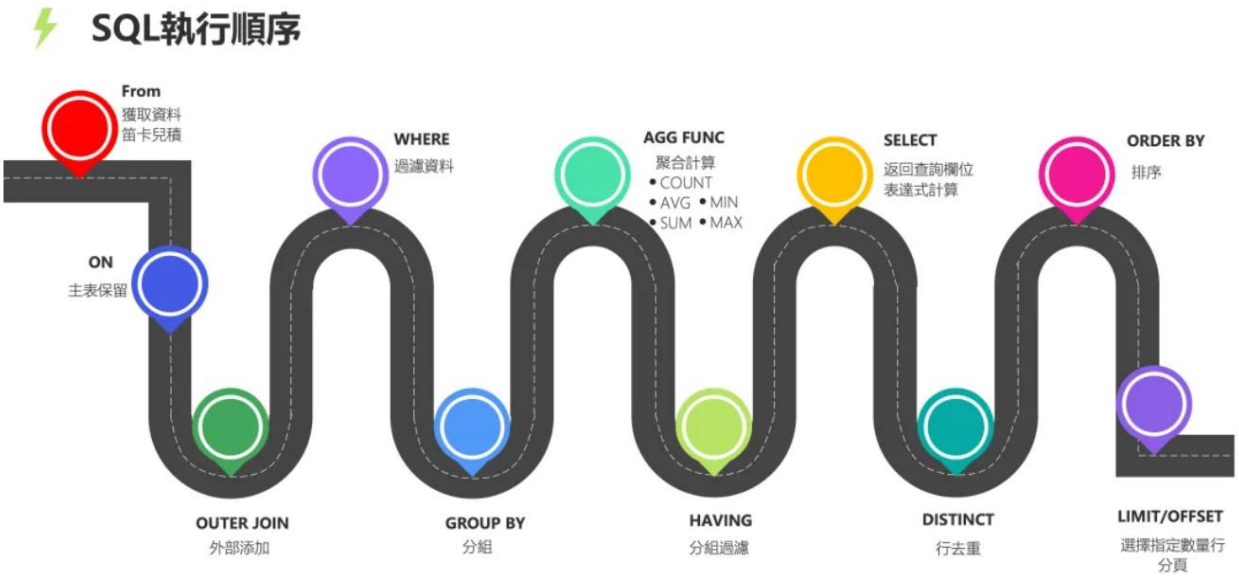
MAX(column)：返回指定列的最大值。

```
SELECT MAX(price) AS MaxPrice FROM orders;
```

MIN(column)：返回指定列的最小值。

```
SELECT MIN(price) AS MinPrice FROM orders;
```

SQL 查询语句的执行顺序是怎么样的?



所有的查询语句都是从 FROM 开始执行，在执行过程中，每个步骤都会生成一个虚拟表，这个虚拟表将作为下一个执行步骤的输入，最后一个步骤产生的虚拟表即为输出结果。

```
(9) SELECT
(10) DISTINCT <column>,
(6) AGG_FUNC <column> or <expression>, ...
(1) FROM <left_table>
    (3) <join_type>JOIN<right_table>
    (2) ON<join_condition>
(4) WHERE <where_condition>
(5) GROUP BY <group_by_list>
(7) WITH {CUBE|ROLLUP}
(8) HAVING <having_condtion>
(11) ORDER BY <order_by_list>
(12) LIMIT <limit_number>;
```

sql 题：给学生表、课程成绩表，求不存在 01 课程但存在 02 课程的学生的成绩

可以使用 SQL 的子查询和 `LEFT JOIN` 或者 `EXISTS` 关键字来实现，这里我将展示两种不同的方法来完成这个查询。

假设我们有以下两张表：

1. `Student` 表，其中包含学生的 `sid`（学生编号）和其他相关信息。
2. `Score` 表，其中包含 `sid`（学生编号），`cid`（课程编号）和 `score`（分数）。

方法 1：使用 LEFT JOIN 和 IS NULL

```
SELECT s.sid, s.sname, sc2.cid, sc2.score
FROM Student s
LEFT JOIN Score AS sc1 ON s.sid = sc1.sid AND sc1.cid = '01'
LEFT JOIN Score AS sc2 ON s.sid = sc2.sid AND sc2.cid = '02'
WHERE sc1.cid IS NULL AND sc2.cid IS NOT NULL;
```

方法 2：使用 NOT EXISTS

```
SELECT s.sid, s.sname, sc.cid, sc.score
FROM Student s
JOIN Score sc ON s.sid = sc.sid AND sc.cid = '02'
```

```
WHERE NOT EXISTS (
    SELECT 1 FROM Score sc1 WHERE sc1.sid = s.sid AND sc1.cid = '01'
);
```

给定一个学生表 student_score（stu_id，subject_id，score），查询总分排名在 5-10 名的学生 id 及对应的总分

可以使用以下 SQL 查询来检索总分排名在 5 到 10 名的学生 ID 及对应的总分。其中我们先计算每个学生的总分，然后为其分配一个排名，最后检索排名在 5 到 10 之间的记录。

```
WITH StudentTotalScores AS (
    SELECT
        stu_id,
        SUM(score) AS total_score
    FROM
        student_score
    GROUP BY
        stu_id
),
RankedStudents AS (
    SELECT
        stu_id,
        total_score,
        RANK() OVER (ORDER BY total_score DESC) AS ranking
    FROM
        StudentTotalScores
)
SELECT
    stu_id,
    total_score
FROM
    RankedStudents
WHERE
    ranking BETWEEN 5 AND 10;
```

解释：

1. 子查询 StudentTotalScores 中，我们通过对 student_score 表中的 stu_id 分组来计算每个学生的总分。
2. 子查询 RankedStudents 中，我们使用 RANK() 函数为每个学生分配一个排名，按总分从高到低排序。
3. 最后，我们在主查询中选择排名在 5 到 10 之间的学生。

SQL 题：查某个班级下所有学生的选课情况

有三张表：学生信息表、学生选课表、学生班级表

学生信息表（students）结构如下：

```
CREATE TABLE students (
  student_id INT PRIMARY KEY, //学生的唯一标识，主键。
  student_name VARCHAR(50), //学生姓名。
  class_id INT //学生所属班级的标识，用于关联班级表。
);
```

学生选课表（`course_selections`）结构如下：

```
CREATE TABLE course_selections (
    selection_id INT PRIMARY KEY, //选课记录的唯一标识，主键。
    student_id INT, //选课学生的标识，用于关联学生信息表。
    course_name VARCHAR(50), //所选课程的名称。
);
```

学生班级表（`classes`）结构如下：

```
CREATE TABLE classes (
    class_id INT PRIMARY KEY, //班级的唯一标识，主键。
    class_name VARCHAR(50) //班级名称。
);
```

要查询某个班级（例如班级名称为 'Class A'）下所有学生的选课情况，可以使用以下 SQL 查询语句：

```
SELECT
    s.student_id,
```

```
    s.student_name,
    cs.course_name
FROM
    students s
JOIN
    course_selections cs ON s.student_id = cs.student_id
JOIN
    classes c ON s.class_id = c.class_id
WHERE
    c.class_name = 'Class A';
```

如何用 MySQL 实现一个可重入的锁?

创建一个保存锁记录的表：

```
CREATE TABLE `lock_table` (
    `id` INT AUTO_INCREMENT PRIMARY KEY,
    //该字段用于存储锁的名称，作为锁的唯一标识符。
    `lock_name` VARCHAR(255) NOT NULL,
    // holder_thread 该字段存储当前持有锁的线程的名称，用于标识哪个线程持有该锁。
    `holder_thread` VARCHAR(255),
    // reentry_count 该字段存储锁的重入次数，用于实现锁的可重入性
    `reentry_count` INT DEFAULT 0
);
```

加锁的实现逻辑

1. 开启事务
2. 执行 SQL SELECT holder_thread, reentry_count FROM lock_table WHERE lock_name =? FOR UPDATE，查询是否存在该记录：
    - 如果记录不存在，则直接加锁，执行 INSERT INTO lock_table (lock_name, holder_thread, reentry_count) VALUES (?,?, 1)
    - 如果记录存在，且持有者是同一个线程，则可冲入，增加重入次数，执行 UPDATE lock_table SET reentry_count = reentry_count + 1 WHERE lock_name =?
3. 提交事务

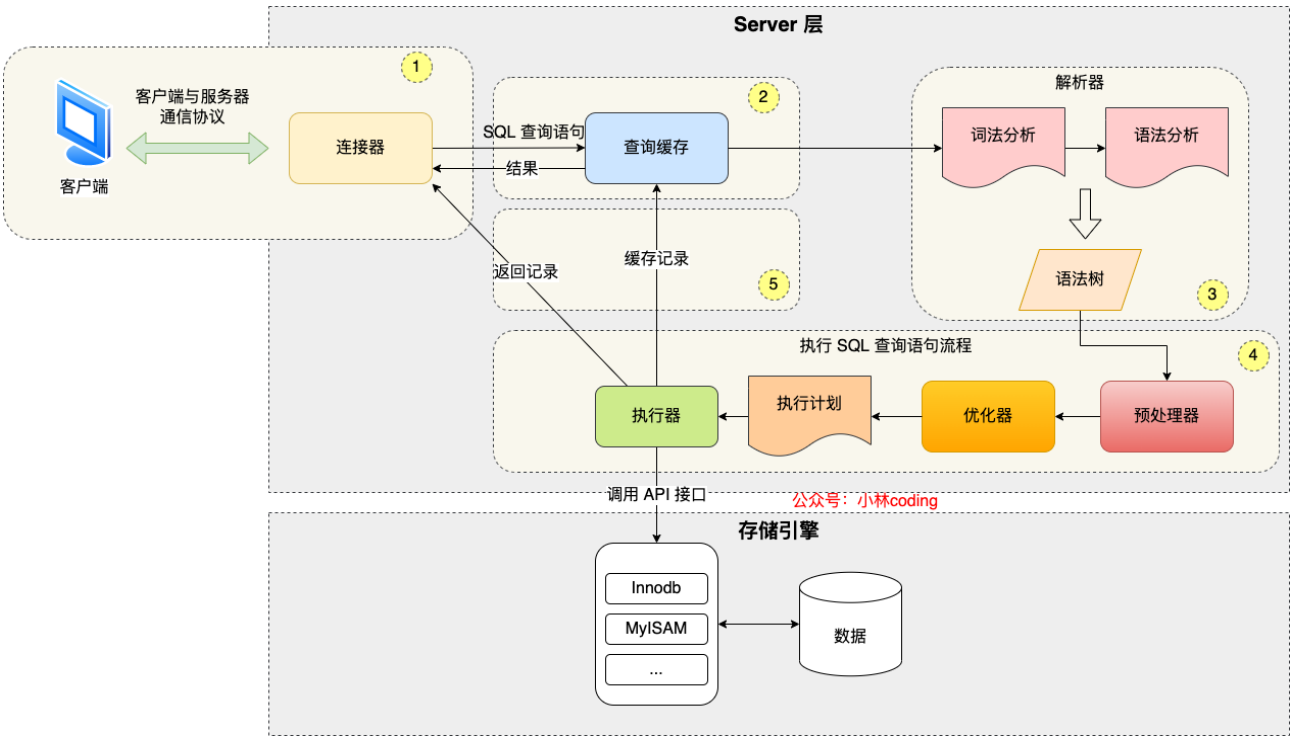
解锁的逻辑：

1. 开启事务
2. 执行 SQL SELECT holder_thread, reentry_count FROM lock_table WHERE lock_name =? FOR UPDATE，查询是否存在该记录：
    - 如果记录存在，且持有者是同一个线程，且可重入数大于 1 ，则减少重入次数 UPDATE lock_table SET reentry_count = reentry_count - 1 WHERE lock_name =?
    - 如果记录存在，且持有者是同一个线程，且可重入数小于等于 0 ，则完全释放锁，DELETE FROM lock_table WHERE lock_name =?
3. 提交事务

存储引擎

执行一条 SQL 请求的过程是什么？

先来一个上帝视角图，下面就是 MySQL 执行一条 SQL 查询语句的流程，也从图中可以看到 MySQL 内部架构里的各个功能模块。



- 连接器：建立连接，管理连接、校验用户身份；
- 查询缓存：查询语句如果命中查询缓存则直接返回，否则继续往下执行。MySQL 8.0 已删除该模块；

- 解析 SQL，通过解析器对 SQL 查询语句进行词法分析、语法分析，然后构建语法树，方便后续模块读取表名、字段、语句类型；
- 执行 SQL：执行 SQL 共有三个阶段：
    - 预处理阶段：检查表或字段是否存在；将 `select *` 中的 `*` 符号扩展为表上的所有列。
    - 优化阶段：基于查询成本的考虑， 选择查询成本最小的执行计划；
    - 执行阶段：根据执行计划执行 SQL 查询语句，从存储引擎读取记录，返回给客户端；

讲一讲 mysql 的引擎吧，你有什么了解?

- InnoDB：InnoDB 是 MySQL 的默认存储引擎，具有 ACID 事务支持、行级锁、外键约束等特性。它适用于高并发的读写操作，支持较好的数据完整性和并发控制。
- MyISAM：MyISAM 是 MySQL 的另一种常见的存储引擎，具有较低的存储空间和内存消耗，适用于大量读操作的场景。然而，MyISAM 不支持事务、行级锁和外键约束，因此在并发写入和数据完整性方面有一定的限制。
- Memory：Memory 引擎将数据存储在内存中，适用于对性能要求较高的读操作，但是在服务器重启或崩溃时数据会丢失。它不支持事务、行级锁和外键约束。

MySQL 为什么 InnoDB 是默认引擎?

InnoDB 引擎在事务支持、并发性能、崩溃恢复等方面具有优势，因此被 MySQL 选择为默认的存储引擎。

- 事务支持：InnoDB 引擎提供了对事务的支持，可以进行 ACID（原子性、一致性、隔离性、持久性）属性的操作。Myisam 存储引擎是不支持事务的。
- 并发性能：InnoDB 引擎采用了行级锁定的机制，可以提供更好的并发性能，Myisam 存储引擎只支持表锁，锁的粒度比较大。
- 崩溃恢复：InnoDB 引引擎通过 redolog 日志实现了崩溃恢复，可以在数据库发生异常情况（如断电）时，通过日志文件进行恢复，保证数据的持久性和一致性。Myisam 是不支持崩溃恢复的。

说一下 mysql 的 innodb 与 MyISAM 的区别?

- **事务**：InnoDB 支持事务，MyISAM 不支持事务，这是 MySQL 将默认存储引擎从 MyISAM 变成 InnoDB 的重要原因之一。

- **索引结构**：InnoDB 是聚簇索引，MyISAM 是非聚簇索引。聚簇索引的文件存放在主键索引的叶子节点上，因此 InnoDB 必须要有主键，通过主键索引效率很高。但是辅助索引需要两次查询，先查询到主键，然后再通过主键查询到数据。因此，主键不应该过大，因为主键太大，其他索引也都会很大。而 MyISAM 是非聚簇索引，数据文件是分离的，索引保存的是数据文件的指针。主键索引和辅助索引是独立的。
- **锁粒度**：InnoDB 最小的锁粒度是行锁，MyISAM 最小的锁粒度是表锁。一个更新语句会锁住整张表，导致其他查询和更新都会被阻塞，因此并发访问受限。
- **count 的效率**：InnoDB 不保存表的具体行数，执行 select count(*) from table 时需要全表扫描。而 MyISAM 用一个变量保存了整个表的行数，执行上述语句时只需要读出该变量即可，速度很快。

数据管理里，数据文件大体分成哪几种数据文件?

我们每创建一个 database（数据库） 都会在 /var/lib/mysql/ 目录里面创建一个以 database 为名的目录，然后保存表结构和表数据的文件都会存放在这个目录里。

比如，我这里有一个名为 my_test 的 database，该 database 里有一张名为 t_order 数据库表。



然后，我们进入 /var/lib/mysql/my_test 目录，看看里面有什么文件？

```
[root@xiaolin ~]#ls /var/lib/mysql/my_test
db.opt
t_order.frm
t_order.ibd
```

可以看到，共有三个文件，这三个文件分别代表着：

- db.opt，用来存储当前数据库的默认字符集和字符校验规则。
- t_order.frm ，t_order 的表结构会保存在这个文件。在 MySQL 中建立一张表都会生成一个.frm 文件，该文件是用来保存每个表的元数据信息的，主要包含表结构定义。

- t_order.ibd，t_order 的表数据会保存在这个文件。表数据既可以存在共享表空间文件（文件名：ibdata1）里，也可以存放在独占表空间文件（文件名：表名字.ibd）。这个行为是由参数 innodb_file_per_table 控制的，若设置了参数 innodb_file_per_table 为 1，则会将存储的数据、索引等信息单独存储在一个独占表空间，从 MySQL 5.6.6 版本开始，它的默认值就是 1 了，因此从这个版本之后， MySQL 中每一张表的数据都存放在一个独立的 .ibd 文件。

索引

索引是什么？有什么好处?

索引类似于书籍的目录，可以减少扫描的数据量，提高查询效率。

- 如果查询的时候，没有用到索引就会全表扫描，这时候查询的时间复杂度是 On
- 如果用到了索引，那么查询的时候，可以基于二分查找算法，通过索引快速定位到目标数据， mysql 索引的数据结构一般是 b+树，其搜索复杂度为 O(logdN)，其中 d 表示节点允许的最大子节点个数为 d 个。

讲讲索引的分类是什么?

MySQL 可以按照四个角度来分类索引。

- 按「数据结构」分类：**B+tree 索引、Hash 索引、Full-text 索引**。
- 按「物理存储」分类：**聚簇索引（主键索引）、二级索引（辅助索引）**。
- 按「字段特性」分类：**主键索引、唯一索引、普通索引、前缀索引**。
- 按「字段个数」分类：**单列索引、联合索引**。

接下来，按照这些角度来说说各类索引的特点。

> 按数据结构分类

从数据结构的角度来看，MySQL 常见索引有 B+Tree 索引、HASH 索引、Full-Text 索引。

每一种存储引擎支持的索引类型不一定相同，我在表中总结了 MySQL 常见的存储引擎 InnoDB、MyISAM 和 Memory 分别支持的索引类型。

索引类型	InnoDB 引擎	MyISAM 引擎	Memory 引擎
B+Tree 索引	Yes	Yes	Yes
HASH 索引	No (不支持hash索引，但是在内存结构中有一个自适应hash索引)	No	Yes
Full-Text 索引	Yes (MySQL 5.6 版本后支持)	Yes	No

InnoDB 是在 MySQL 5.5 之后成为默认的 MySQL 存储引擎，B+Tree 索引类型也是 MySQL 存储引擎采用最多的索引类型。

在创建表时，InnoDB 存储引擎会根据不同的场景选择不同的列作为索引：

- 如果有主键，默认会使用主键作为聚簇索引的索引键（key）；
- 如果没有主键，就选择第一个不包含 NULL 值的唯一列作为聚簇索引的索引键（key）；
- 在上面两个都没有的情况下，InnoDB 将自动生成一个隐式自增 id 列作为聚簇索引的索引键（key）；

其它索引都属于辅助索引（Secondary Index），也被称为二级索引或非聚簇索引。**创建的主键索引和二级索引默认使用的是 B+Tree 索引**。

> 按物理存储分类

从物理存储的角度来看，索引分为聚簇索引（主键索引）、二级索引（辅助索引）。

这两个区别在前面也提到了：

- 主键索引的 B+Tree 的叶子节点存放的是实际数据，所有完整的用户记录都存放在主键索引的 B+Tree 的叶子节点里；
- 二级索引的 B+Tree 的叶子节点存放的是主键值，而不是实际数据。

所以，在查询时使用了二级索引，如果查询的数据能在二级索引里查询的到，那么就不需要回表，这个过程就是覆盖索引。如果查询的数据不在二级索引里，就会先检索二级索引，找到对应的叶子节点，获取到主键值后，然后再检索主键索引，就能查询到数据了，这个过程就是回表。

> 按字段特性分类

从字段特性的角度来看，索引分为主键索引、唯一索引、普通索引、前缀索引。

- 主键索引

主键索引就是建立在主键字段上的索引，通常在创建表的时候一起创建，一张表最多只有一个主键索引，索引列的值不允许有空值。

在创建表时，创建主键索引的方式如下：

```
CREATE TABLE table_name  (
  ....
  PRIMARY KEY (index_column_1) USING BTREE
);
```

- 唯一索引

唯一索引建立在 UNIQUE 字段上的索引，一张表可以有多个唯一索引，索引列的值必须唯一，但是允许有空值。

在创建表时，创建唯一索引的方式如下：

```
CREATE TABLE table_name  (
  ....
  UNIQUE KEY(index_column_1,index_column_2,...)
);
```

建表后，如果要创建唯一索引，可以使用这面这条命令：

```
CREATE UNIQUE INDEX index_name
ON table_name(index_column_1,index_column_2,...);
```

- 普通索引

普通索引就是建立在普通字段上的索引，既不要求字段为主键，也不要求字段为 UNIQUE。

在创建表时，创建普通索引的方式如下：

```
CREATE TABLE table_name  (
  ....
  INDEX(index_column_1,index_column_2,...)
```

```
);
```

建表后，如果要创建普通索引，可以使用这面这条命令：

```
CREATE INDEX index_name
ON table_name(index_column_1,index_column_2,...);
```

- 前缀索引

前缀索引是指对字符类型字段的前几个字符建立的索引，而不是在整个字段上建立的索引，前缀索引可以建立在字段类型为 char、 varchar、binary、varbinary 的列上。

使用前缀索引的目的是为了减少索引占用的存储空间，提升查询效率。

在创建表时，创建前缀索引的方式如下：

```
CREATE TABLE table_name(
    column_list,
    INDEX(column_name(length))
);
```

建表后，如果要创建前缀索引，可以使用这面这条命令：

```
CREATE INDEX index_name
ON table_name(column_name(length));
```

> 按字段个数分类

从字段个数的角度来看，索引分为单列索引、联合索引（复合索引）。

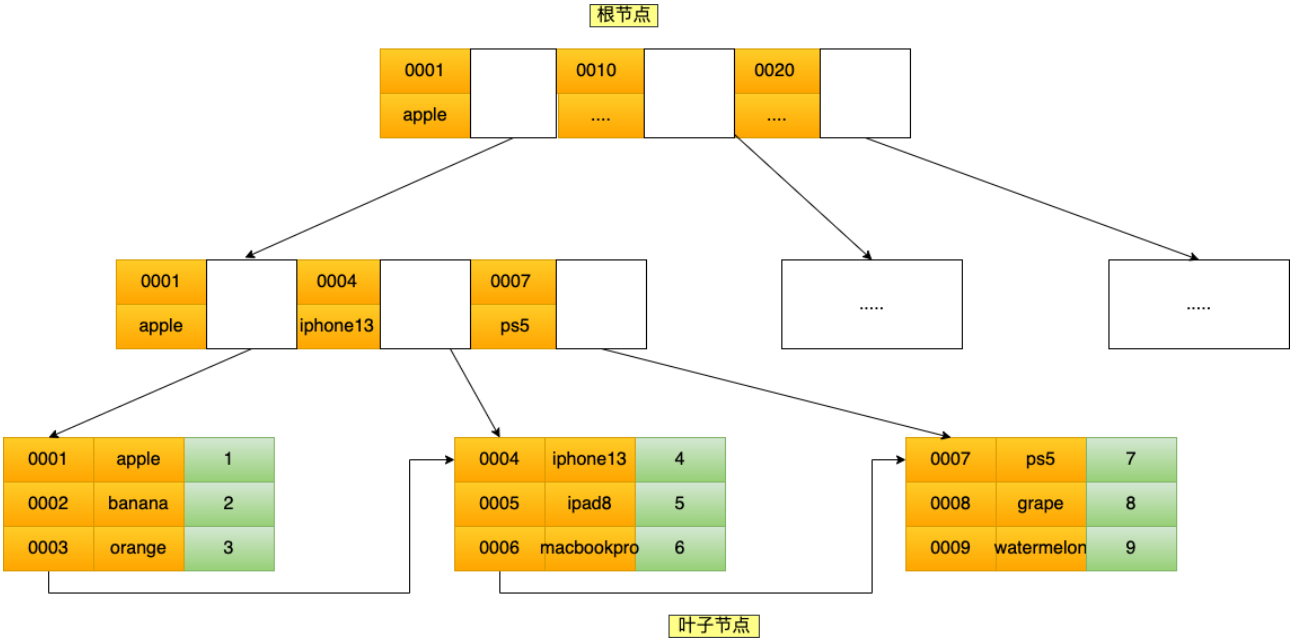
- 建立在单列上的索引称为单列索引，比如主键索引；
- 建立在多列上的索引称为联合索引；

通过将多个字段组合成一个索引，该索引就被称为联合索引。

比如，将商品表中的 product_no 和 name 字段组合成联合索引(product_no, name)，创建联合索引的方式如下：

```
CREATE INDEX index_product_no_name ON product(product_no, name);
```

联合索引(product_no, name) 的 B+Tree 示意图如下（图中叶子节点之间我画了单向链表，但是实际上是双向链表，原图我找不到了，修改不了，偷个懒我不重画了，大家脑补成双向链表就行）。



可以看到，联合索引的非叶子节点用两个字段的值作为 B+Tree 的 key 值。当在联合索引查询数据时，先按 product_no 字段比较，在 product_no 相同的情况下再按 name 字段比较。

也就是说，联合索引查询的 B+Tree 是先按 product_no 进行排序，然后再 product_no 相同的情况再按 name 字段排序。

因此，使用联合索引时，存在**最左匹配原则**，也就是按照最左优先的方式进行索引的匹配。在使用联合索引进行查询的时候，如果不遵循「最左匹配原则」，联合索引会失效，这样就无法利用到索引快速查询的特性了。

比如，如果创建了一个 (a, b, c) 联合索引，如果查询条件是以下这几种，就可以匹配上联合索引：

- where a=1;
- where a=1 and b=2 and c=3;
- where a=1 and b=2;

需要注意的是，因为有查询优化器，所以 a 字段在 where 子句的顺序并不重要。

但是，如果查询条件是以下这几种，因为不符合最左匹配原则，所以就无法匹配上联合索引，联合索引就会失效:

- where b=2;
- where c=3;
- where b=2 and c=3;

上面这些查询条件之所以会失效，是因为(a，b，c）联合索引，是先按 a 排序，在 a 相同的情况再按 b 排序，在 b 相同的情况再按 c 排序。所以，**b 和 c 是全局无序，局部相对有序的**，这样在没有遵循最左匹配原则的情况下，是无法利用到索引的。

联合索引有一些特殊情况，**并不是查询过程使用了联合索引查询，就代表联合索引中的所有字段都用到了联合索引进行索引查询**，也就是可能存在部分字段用到联合索引的 B+Tree，部分字段没有用到联合索引的 B+Tree 的情况。

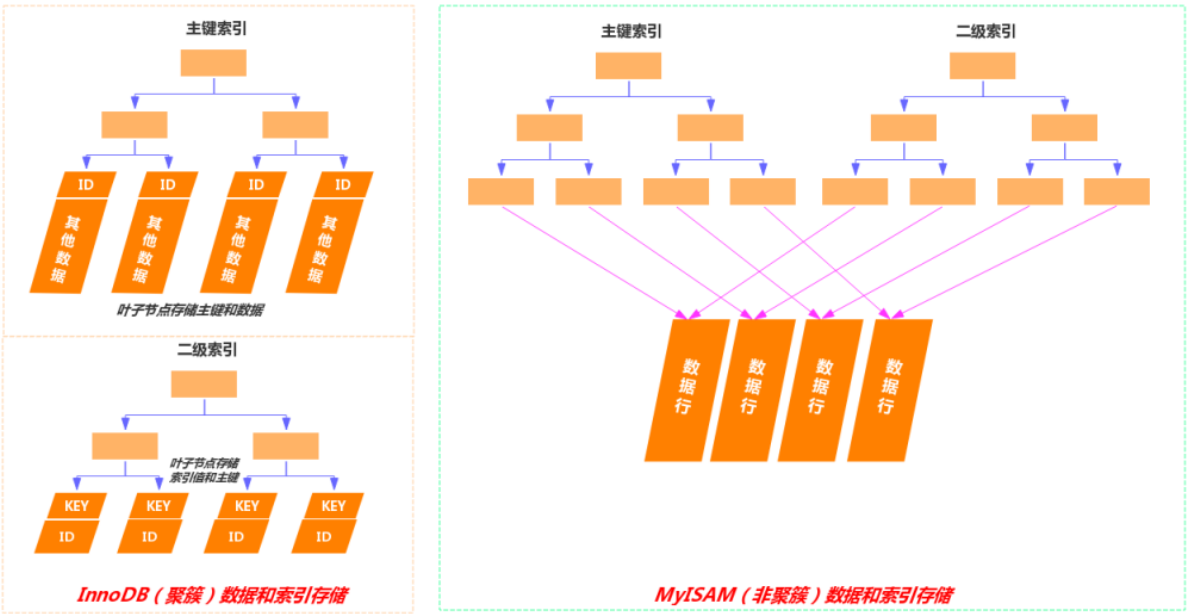
这种特殊情况就发生在范围查询。联合索引的最左匹配原则会一直向右匹配直到遇到「范围查询」就会停止匹配。**也就是范围查询的字段可以用到联合索引，但是在范围查询字段的后面的字段无法用到联合索引**。

哈希索引使用场景是什么?

哈希索引优势是等值查询极快、能直接通过哈希函数定位数据位置，而短板是不支持范围查询、排序和前缀匹配，所以使用场景必然是那些以等值查询为核心，且不需要复杂查询操作的场景。

比如 MySQL 的 Memory 存储引擎，默认就用哈希索引。因为内存表本身是基于内存存储的，追求极致的查询速度，而这类场景通常都是简单的等值查询，比如缓存用户登录信息，通过用户 ID 快速查找对应的登录状态、权限信息，这时候哈希索引能直接通过用户 ID 计算哈希值，定位到数据所在的桶，查询效率接近 O(1)，比 B+ 树索引的等值查询更快。

MySQL 聚簇索引和非聚簇索引的区别是什么?



- **数据存储**：在聚簇索引中，数据行按照索引键值的顺序存储，也就是说，索引的叶子节点包含了实际的数据行。这意味着索引结构本身就是数据的物理存储结构。非聚簇索引的叶子节点不包含完整的数据行，而是包含指向数据行的指针或主键值。数据行本身存储在聚簇索引中。
- **索引与数据关系**：由于数据与索引紧密相连，当通过聚簇索引查找数据时，可以直接从索引中获得数据行，而不需要额外的步骤去查找数据所在的位置。当通过非聚簇索引查找数据时，首先在非聚簇索引中找到对应的主键值，然后通过这个主键值回溯到聚簇索引中查找实际的数据行，这个过程称为“回表”。
- **唯一性**：聚簇索引通常是基于主键构建的，因此每个表只能有一个聚簇索引，因为数据只能有一种物理排序方式。一个表可以有多个非聚簇索引，因为它们不直接影响数据的物理存储位置。
- **效率**：对于范围查询和排序查询，聚簇索引通常更有效率，因为它避免了额外的寻址开销。非聚簇索引在使用覆盖索引进行查询时效率更高，因为它不需要读取完整的数据行。但是需要进行回表的操作，使用非聚簇索引效率比较低，因为需要进行额外的回表操作。

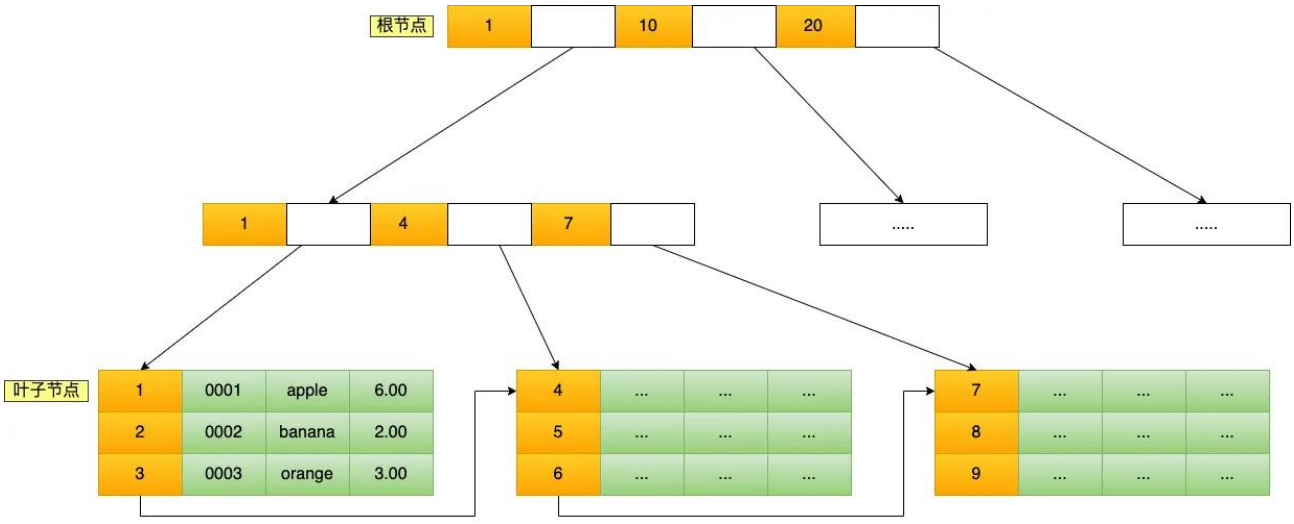
如果聚簇索引的数据更新，它的存储要不要变化?

- 如果更新的数据是非索引数据，也就是普通的用户记录，那么存储结构是不会发生变化
- 如果更新的数据是索引数据，那么存储结构是有变化的，因为要维护 b+树的有序性

MySQL 主键是聚簇索引吗?

在 MySQL 的 InnoDB 存储引擎中，主键确实是以聚簇索引的形式存储的。

InnoDB 将数据存储在 B+树的结构中，其中主键索引的 B+树就是所谓的聚簇索引。这意味着表中的数据行在物理上是按照主键的顺序排列的，聚簇索引的叶节点包含了实际的数据行。



InnoDB 在创建聚簇索引时，会根据不同的场景选择不同的列作为索引：

- 如果有主键，默认会使用主键作为聚簇索引的索引键；
- 如果没有主键，就选择第一个不包含 NULL 值的唯一列作为聚簇索引的索引键；
- 在上面两个都没有的情况下，InnoDB 将自动生成一个隐式自增 id 列作为聚簇索引的索引键；

一张表只能有一个聚簇索引，那为了实现非主键字段的快速搜索，就引出了二级索引（非聚簇索引/辅助索引），它也是利用了 B+ 树的数据结构，但是二级索引的叶子节点存放的是主键值，不是实际数据。

什么字段适合当做主键?

- 字段具有唯一性，且不能为空的特性
- 字段最好的是有递增的趋势的，如果字段的值是随机无序的，可能会引发页分裂的问题，造型性能影响。
- 不建议用业务数据作为主键，比如会员卡号、订单号、学生号之类的，因为我们无法预测未来会不会因为业务需要，而出现业务字段重复或者重用的情况。
- 通常情况下会用自增字段来做主键，对于单机系统来说是没问题的。但是，如果有多台服务器，各自都可以录入数据，那就不一定适用了。因为如果每台机器各自产生的数据需要合并，就可能会出现主键重复的问题，这时候就需要考虑分布式 id 的方案了。

性别字段能加索引么？为啥?

不建议针对性别字段加索引。

实际上与索引创建规则之一区分度有关，性别字段假设有 100w 数据，50w 男、50w 女，区别度几乎等于 0 。

区分度的计算方式 ： select count(DISTINCT sex)/count(*) from sys_user

实际上对于性别字段不适合创建索引，是因为 select * 操作，还得进行 50w 次回表操作，根据主键从聚簇索引中找到其他字段 ，这一部分开销从上面的测试来说还是比较大的，所以从性能角度来看不建议性别字段加索引，加上索引并不是索引失效，而是回表操作使得变慢的。

既然走索引的查询的成本比全表扫描高，优化器就会选择全表扫描的方向进行查询，这时候建立的性别字段索引就没有启到加快查询的作用，反而还因为创建了索引占用了空间。

表中十个字段，你主键用自增 ID 还是 UUID，为什么?

用的是自增 id。

因为 uuid 相对顺序的自增 id 来说是毫无规律可言的，新行的值不一定要比之前的主键的值要大，所以 innodb 无法做到总是把新行插入到索引的最后，而是需要为新行寻找新的合适的位置从而来分配新的空间。

这个过程需要做很多额外的操作，数据的毫无顺序会导致数据分布散乱，将会导致以下的问题：

- 写入的目标页很可能已经刷新到磁盘上并且从缓存上移除，或者还没有被加载到缓存中，innodb 在插入之前不得不先找到并从磁盘读取目标页到内存中，这将导致大量的随机 IO。
- 因为写入是乱序的，innodb 不得不频繁的做页分裂操作，以便为新的行分配空间，页分裂导致移动大量的数据，影响性能。
- 由于频繁的页分裂，页会变得稀疏并被不规则的填充，最终会导致数据会有碎片。

结论：使用 InnoDB 应该尽可能的按主键的自增顺序插入，并且尽可能使用单调的增加的聚簇键的值来插入新行。

什么自增 ID 更快一些，UUID 不快吗，它在 B+树里面存储是有序的吗?

自增的主键的值是顺序的，所以 Innodb 把每一条记录都存储在一条记录的后面，所以自增 id 更快的原因：

- 下一条记录就会写入新的页中，一旦数据按照这种顺序的方式加载，主键页就会近乎于顺序的记录填满，提升了页面的最大填充率，不会有页的浪费
- 新插入的行一定会在原有的最大数据行下一行，mysql 定位和寻址很快，不会为计算新行的位置而做出额外的消耗
- 减少了页分裂和碎片的产生

但是 UUID 不是递增的，MySQL 中索引的数据结构是 B+Tree，这种数据结构的特点是索引树上的节点的数据是有序的，而如果使用 UUID 作为主键，那么每次插入数据时，因为无法保证每次产生的 UUID 有序，所以就会出现新的 UUID 需要插入到索引树的中间去，这样可能会频繁地导致页分裂，使性能下降。

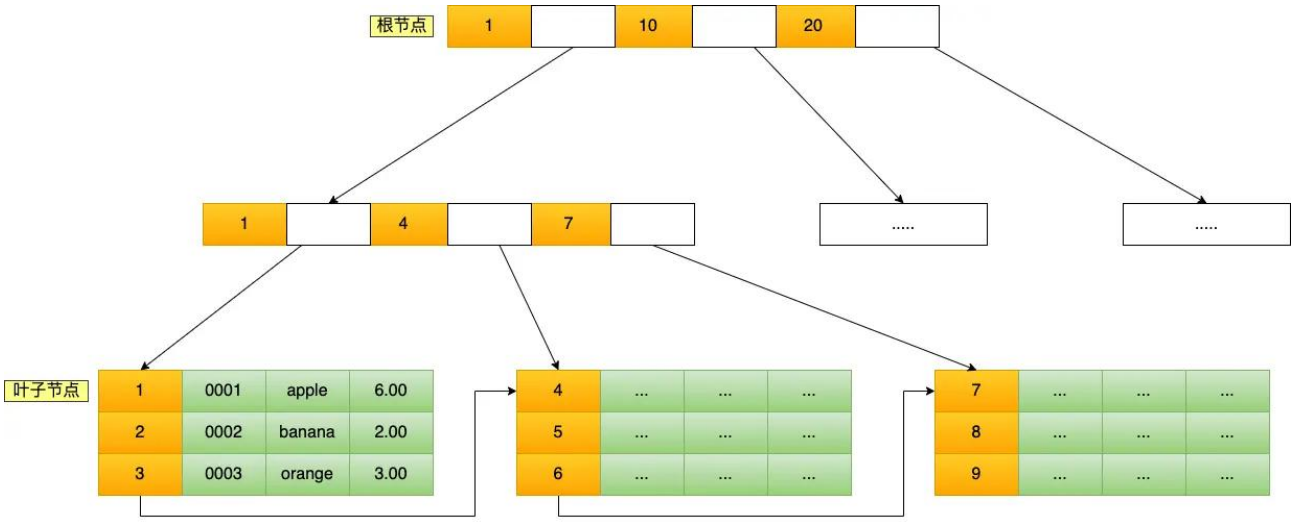
而且，UUID 太占用内存。每个 UUID 由 36 个字符组成，在字符串进行比较时，需要从前往后比较，字符串越长，性能越差。另外字符串越长，占用的内存越大，由于页的大小是固定的，这样一个页上能存放的关键字数量就会越少，这样最终就会导致索引树的高度越大，在索引搜索的时候，发生的磁盘 IO 次数越多，性能越差。

Mysql 中的索引是怎么实现的？

MySQL InnoDB 引擎是用了 B+树作为了索引的数据结构。

B+Tree 是一种多叉树，叶子节点才存放数据，非叶子节点只存放索引，而且每个节点里的数据是**按主键顺序存放**的。每一层父节点的索引值都会出现在下层子节点的索引值中，因此在叶子节点中，包括了所有的索引值信息，并且每一个叶子节点都有两个指针，分别指向下一个叶子节点和上一个叶子节点，形成一个双向链表。

主键索引的 B+Tree 如图所示：



比如，我们执行了下面这条查询语句：

```
select * from product where id= 5;
```

这条语句使用了主键索引查询 id 号为 5 的商品。查询过程是这样的，B+Tree 会自顶向下逐层进行查找：

- 将 5 与根节点的索引数据（1，10，20）比较，5 在 1 和 10 之间，所以根据 B+Tree 的搜索逻辑，找到第二层的索引数据（1，4，7）；
- 在第二层的索引数据（1，4，7)中进行查找，因为 5 在 4 和 7 之间，所以找到第三层的索引数据（4，5，6）；
- 在叶子节点的索引数据（4，5，6）中进行查找，然后我们找到了索引值为 5 的行数据。

数据库的索引和数据都是存储在硬盘的，我们可以把读取一个节点当作一次磁盘 I/O 操作。那么上面的整个查询过程一共经历了 3 个节点，也就是进行了 3 次 I/O 操作。

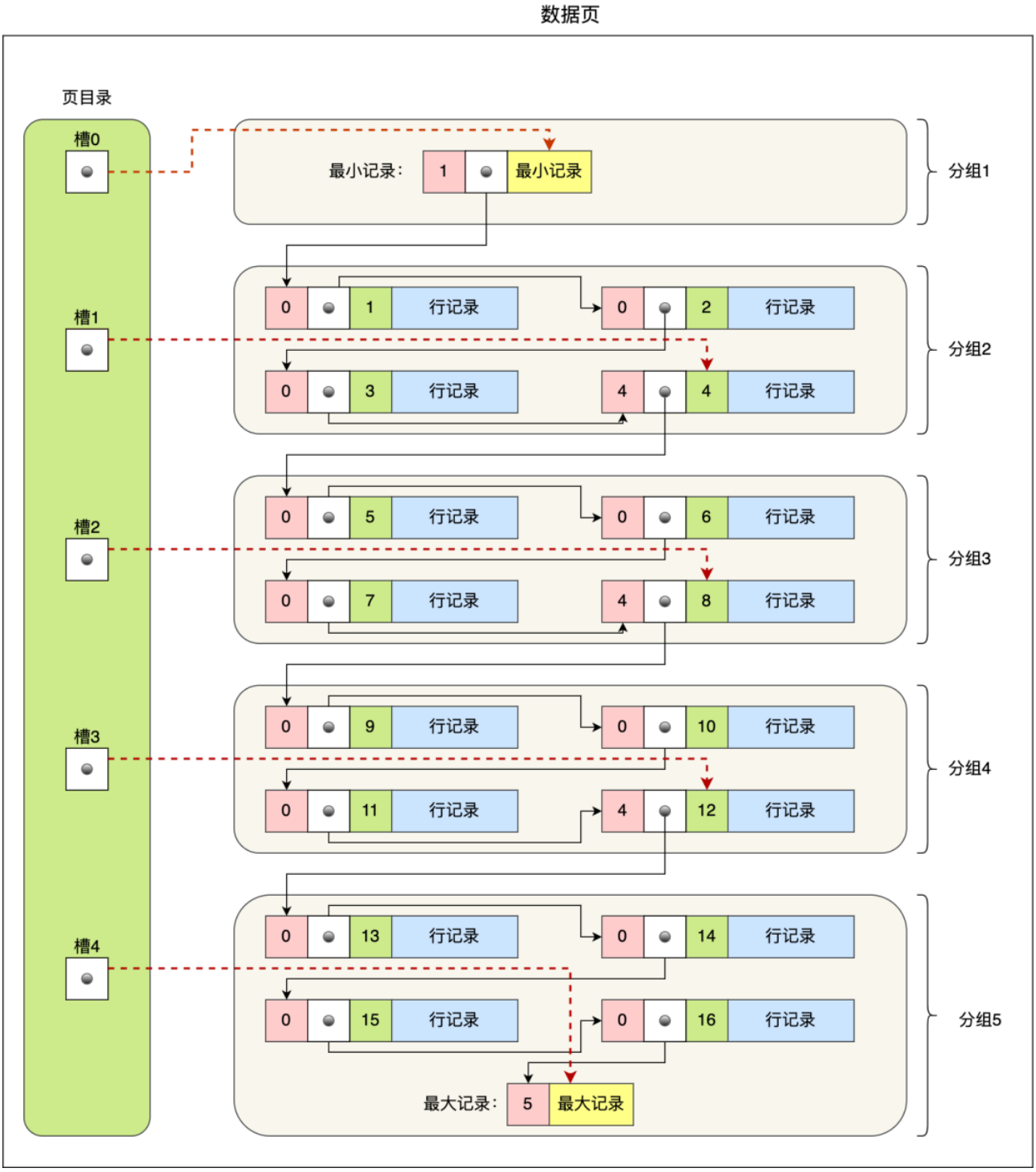
B+Tree 存储千万级的数据只需要 3-4 层高度就可以满足，这意味着从千万级的表查询目标数据最多需要 3-4 次磁盘 I/O，所以**B+Tree 相比于 B 树和二叉树来说，最大的优势在于查询效率很高，因为即使在数据量很大的情况，查询一个数据的磁盘 I/O 依然维持在 3-4 次。**

查询数据时，到了 B+树的叶子节点，之后的查找数据是如何做?

数据页中的记录按照「主键」顺序组成单向链表，单向链表的特点就是插入、删除非常方便，但是检索效率不高，最差的情况下需要遍历链表上的所有节点才能完成检索。

因此，数据页中有一个**页目录**，起到记录的索引作用，就像我们书那样，针对书中内容的每个章节设立了一个目录，想看某个章节的时候，可以查看目录，快速找到对应的章节的页数，而数据页中的页目录就是为了能快速找到记录。那 InnoDB 是如何给记录创建页目录的呢？

页目录与记录的关系如下图：



页目录创建的过程如下：

1. 将所有的记录划分成几个组，这些记录包括最小记录和最大记录，但不包括标记为“已删除”的记录；
2. 每个记录组的最后一条记录就是组内最大的那条记录，并且最后一条记录的头信息中会存储该组一共有多少条记录，作为 n_owned 字段（上图中粉红色字段）
3. 页目录用来存储每组最后一条记录的地址偏移量，这些地址偏移量会按照先后顺序存储起来，每组的地址偏移量也被称之为槽（slot），每个槽相当于指针指向了不同组的最后一个记录。

从图可以看到，**页目录就是由多个槽组成的，槽相当于分组记录的索引**。然后，因为记录是按照「主键值」从小到大排序的，所以**我们通过槽查找记录时，可以使用二分法快速定位要查询的记录在哪个槽（哪个记录分组），定位到槽后，再遍历槽内的所有记录，找到对应的记录**，无需从最小记录开始遍历整个页中的记录链表。以上面那张图举个例子，5 个槽的编号分别为 0，1，2，3，4，我想查找主键为 11 的用户记录：

- 先二分得出槽中间位是 (0+4)/2=2 ，2 号槽里最大的记录为 8。因为 11 > 8，所以需要从 2 号槽后继续搜索记录；
- 再使用二分搜索出 2 号和 4 槽的中间位是 (2+4)/2= 3，3 号槽里最大的记录为 12。因为 11 < 12，所以主键为 11 的记录在 3 号槽里；
- 再从 3 号槽指向的主键值为 9 记录开始向下搜索 2 次，定位到主键为 11 的记录，取出该条记录的信息即为我们想要查找的内容。

B+树的特性是什么?

- **所有叶子节点都在同一层**：这是 B+树的一个重要特性，确保了所有数据项的检索都具有相同的 I/O 延迟，提高了搜索效率。每个叶子节点都包含指向相邻叶子节点的指针，形成一个链表，由于叶子节点之间的链接，B+树非常适合进行范围查询和排序扫描。可以沿着叶子节点的链表顺序访问数据，而无需进行多次随机访问。
- **非叶子节点存储键值**：非叶子节点仅存储键值和指向子节点的指针，不包含数据记录。这些键值用于指导搜索路径，帮助快速定位到正确的叶子节点。并且，由于非叶子节点只存放键值，当数据量比较大时，相对于 B 树，B+树的层高更少，查找效率也就更高。
- **叶子节点存储数据记录**：与 B 树不同，B+树的叶子节点存储实际的数据记录或指向数据记录的指针。这意味着每次搜索都会到达叶子节点，才能找到所需数据。
- **自平衡**：B+树在插入、删除和更新操作后会自动重新平衡，确保树的高度保持相对稳定，从而保持良好的搜索性能。每个节点最多可以有 M 个子节点，最少可以有 ceil(M/2)个子节点（除了根节点），这里的 M 是树的阶数。

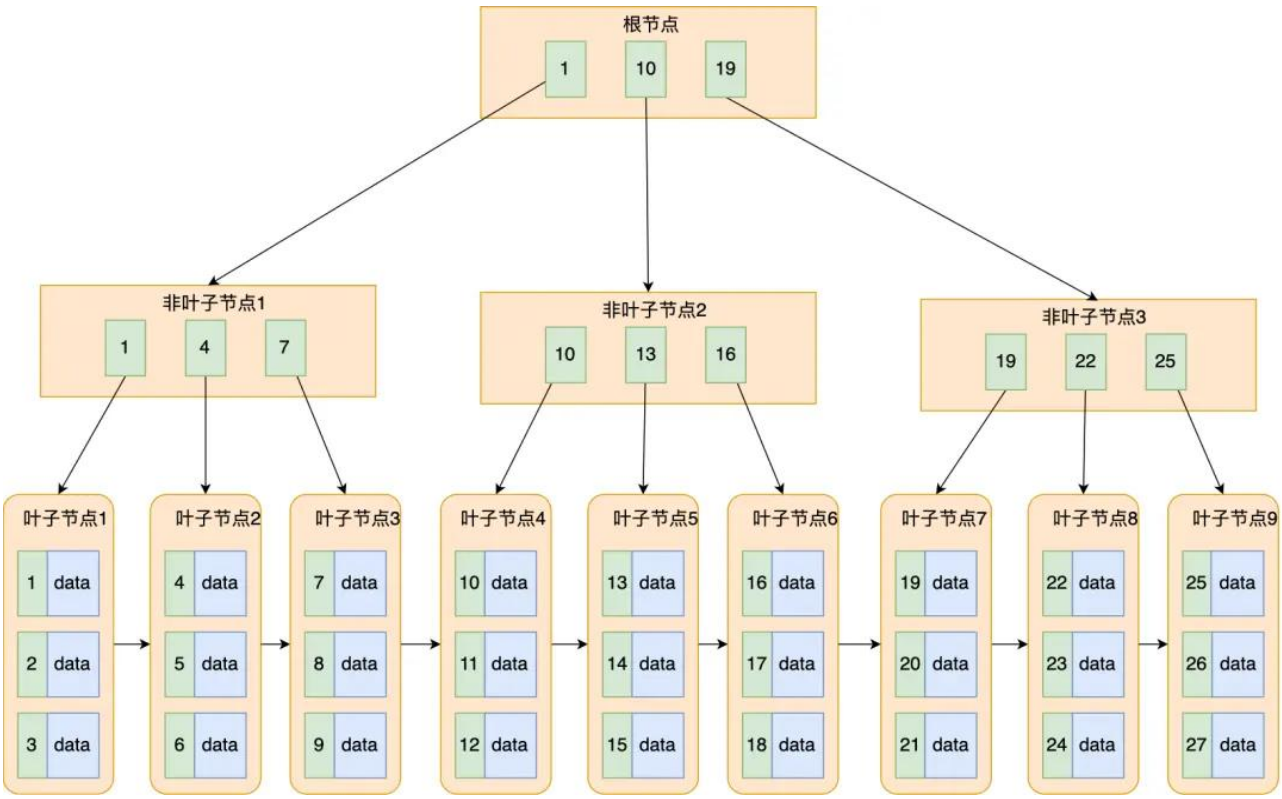
说说 B+树和 B 树的区别

- 在 B+树中，数据都存储在叶子节点上，而非叶子节点只存储索引信息；而 B 树的非叶子节点既存储索引信息也存储部分数据。
- B+树的叶子节点使用链表相连，便于范围查询和顺序访问；B 树的叶子节点没有链表连接。
- B+树的查找性能更稳定，每次查找都需要查找到叶子节点；而 B 树的查找可能会在非叶子节点找到数据，性能相对不稳定。

B+树的好处是什么?

B 树和 B+ 都是通过多叉树的方式，会将树的高度变矮，所以这两个数据结构非常适合检索存于磁盘中的数据。

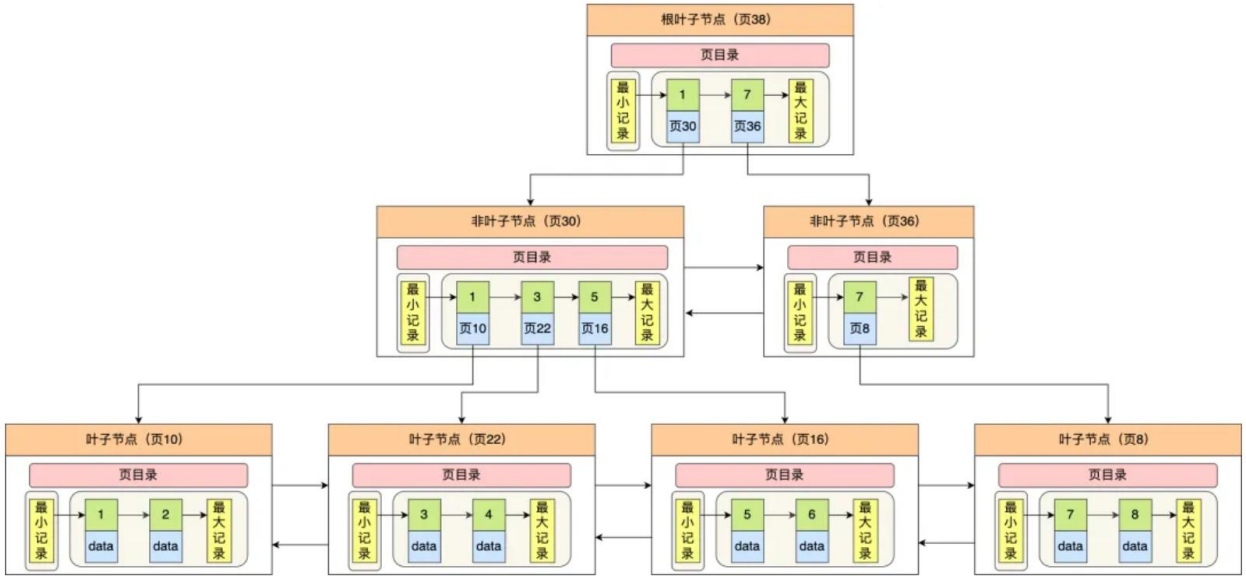
但是 MySQL 默认的存储引擎 InnoDB 采用的是 B+ 作为索引的数据结构，原因有：



- B+ 树的非叶子节点不存放实际的记录数据，仅存放索引，因此数据量相同的情况下，相比存储即存索引又存记录的 B 树，B+树的非叶子节点可以存放更多的索引，因此 B+ 树可以比 B 树更「矮胖」，查询底层节点的磁盘 I/O 次数会更少。
- B+ 树有大量的冗余节点（所有非叶子节点都是冗余索引），这些冗余索引让 B+ 树在插入、删除的效率都更高，比如删除根节点的时候，不会像 B 树那样会发生复杂的树的变化；
- B+ 树叶子节点之间用链表连接了起来，有利于范围查询，而 B 树要实现范围查询，因此只能通过树的遍历来完成范围查询，这会涉及多个节点的磁盘 I/O 操作，范围查询效率不如 B+ 树。

B+树的叶子节点链表是单向还是双向?

双向的，为了实现倒序遍历或者排序。



Innodb 使用的 B+ 树有一些特别的点，比如：

- B+ 树的叶子节点之间是用「双向链表」进行连接，这样的好处是既能向右遍历，也能向左遍历。
- B+ 树点节点内容是数据页，数据页里存放了用户的记录以及各种信息，每个数据页默认大小是 16 KB。

Innodb 根据索引类型不同，分为聚集和二级索引。他们区别在于，聚集索引的叶子节点存放的是实际数据，所有完整的用户记录都存放在聚集索引的叶子节点，而二级索引的叶子节点存放的是主键值，而不是实际数据。

因为表的数据都是存放在聚集索引的叶子节点里，所以 InnoDB 存储引擎一定会为表创建一个聚集索引，且由于数据在物理上只会保存一份，所以聚簇索引只能有一个，而二级索引可以创建多个。

MySQL 为什么用 B+树结构？和其他结构比的优点?

- **B+Tree vs B Tree：**B+Tree 只在叶子节点存储数据，而 B 树 的非叶子节点也要存储数据，所以 B+Tree 的单个节点的数据量更小，在相同的磁盘 I/O 次数下，就能查询更多的节点。另外，B+Tree 叶子节点采用的是双链表连接，适合 MySQL 中常见的基于范围的顺序查找，而 B 树无法做到这一点。
- **B+Tree vs 二叉树：**对于有 N 个叶子节点的 B+Tree，其搜索复杂度为 O(logdN)，其中 d 表示节点允许的最大子节点个数为 d 个。在实际的应用当中， d 值是大于 100 的，这样就保证了，即使数据达到千万级别时，B+Tree 的高度依然维持在 3~4 层左右，也就是说一次数据查询操作只需要做 3~4 次的磁盘 I/O 操作就能查询到目标数据。而二叉树的每个父

节点的儿子节点个数只能是 2 个，意味着其搜索复杂度为 O(logN)，这已经比 B+Tree 高出不少，因此二叉树检索到目标数据所经历的磁盘 I/O 次数要更多。

- **B+Tree vs Hash：**Hash 在做等值查询的时候效率贼快，搜索复杂度为 O(1)。但是 Hash 表不适合做范围查询，它更适合做等值的查询，这也是 B+Tree 索引要比 Hash 表索引有着更广泛的适用场景的原因

为什么 MysSQL 不用 跳表?

B+树的高度在 3 层时存储的数据可能已达千万级别，但对于跳表而言同样去维护千万的数据量那么所造成的跳表层数过高而导致的磁盘 io 次数增多，也就是使用 B+树在存储同样的数据下磁盘 io 次数更少。

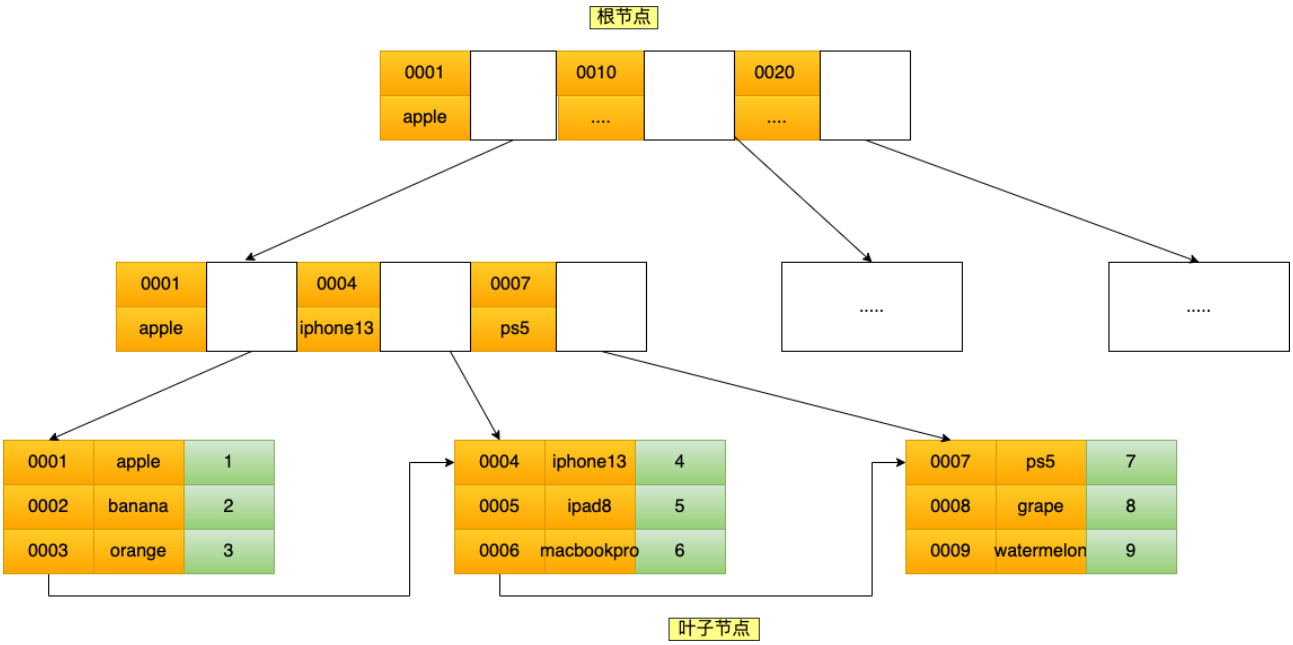
联合索引的实现原理?

将将多个字段组合成一个索引，该索引就被称为联合索引。

比如，将商品表中的 product_no 和 name 字段组合成联合索引(product_no, name)，创建联合索引的方式如下：

```
CREATE INDEX index_product_no_name ON product(product_no, name);
```

联合索引(product_no, name) 的 B+Tree 示意图如下：



可以看到，联合索引的非叶子节点用两个字段的值作为 B+Tree 的 key 值。当在联合索引查询数据时，先按 product_no 字段比较，在 product_no 相同的情况下再按 name 字段比较。

也就是说，联合索引查询的 B+Tree 是先按 product_no 进行排序，然后再 product_no 相同的情况再按 name 字段排序。

因此，使用联合索引时，存在**最左匹配原则**，也就是按照最左优先的方式进行索引的匹配。在使用联合索引进行查询的时候，如果不遵循「最左匹配原则」，联合索引会失效，这样就无法利用到索引快速查询的特性了。

比如，如果创建了一个 (a, b, c) 联合索引，如果查询条件是以下这几种，就可以匹配上联合索引：

- where a=1；
- where a=1 and b=2 and c=3；
- where a=1 and b=2；

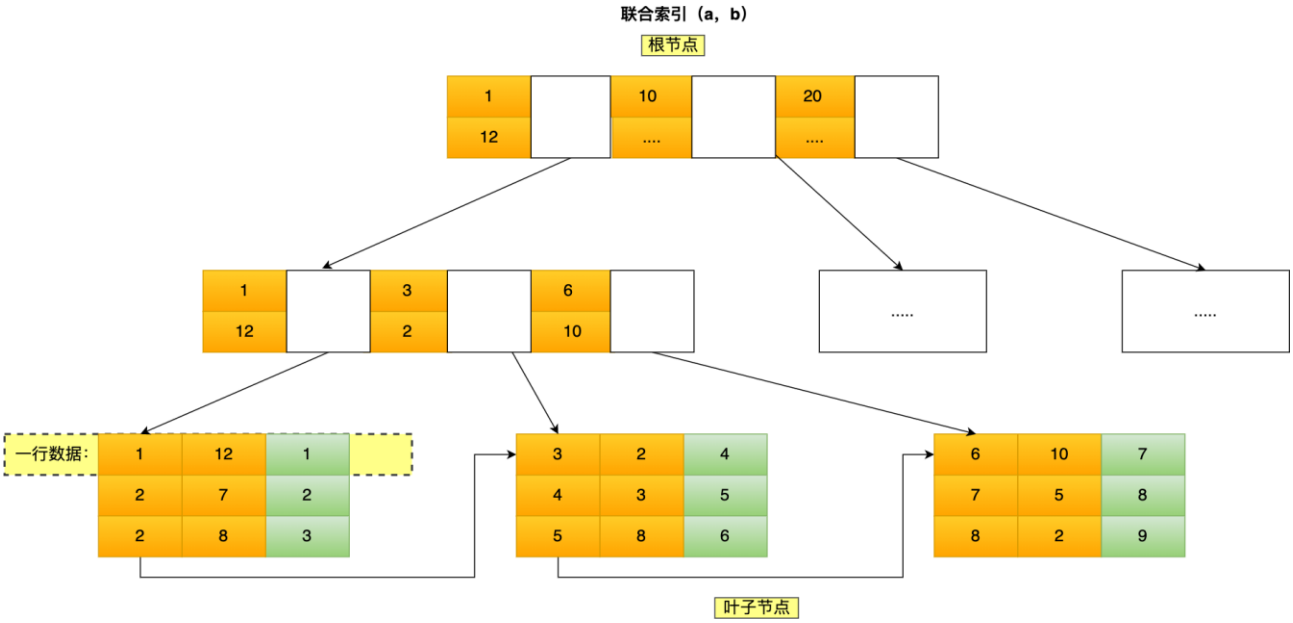
需要注意的是，因为有查询优化器，所以 a 字段在 where 子句的顺序并不重要。

但是，如果查询条件是以下这几种，因为不符合最左匹配原则，所以就无法匹配上联合索引，联合索引就会失效:

- where b=2；
- where c=3；
- where b=2 and c=3；

上面这些查询条件之所以会失效，是因为(a, b, c) 联合索引，是先按 a 排序，在 a 相同的情况再按 b 排序，在 b 相同的情况再按 c 排序。所以，**b 和 c 是全局无序，局部相对有序的**，这样在没有遵循最左匹配原则的情况下，是无法利用到索引的。

我这里举联合索引（a，b）的例子，该联合索引的 B+ Tree 如下：



可以看到，a 是全局有序的（1，2，2，3，4，5，6，7 ,8），而 b 是全局是无序的（12，7，8，2，3，8，10，5，2）。因此，直接执行 where b = 2 这种查询条件没有办法利用联合索引的，**利用索引的前提是索引里的 key 是有序的**。

只有在 a 相同的情况才，b 才是有序的，比如 a 等于 2 的时候，b 的值为（7，8），这时就是有序的，这个有序状态是局部的，因此，执行 where a = 2 and b = 7 是 a 和 b 字段能用到联合索引的，也就是联合索引生效了。

创建联合索引时需要注意什么？

建立联合索引时的字段顺序，对索引效率也有很大影响。越靠前的字段被用于索引过滤的概率越高，实际开发工作中**建立联合索引时，要把区分度大的字段排在前面，这样区分度大的字段越有可能被更多的 SQL 使用到**。

区分度就是某个字段 column 不同值的个数「除以」表的总行数，计算公式如下：

$$区分度 = \frac{distinct(column)}{count(*)}$$

比如，性别的区分度就很小，不适合建立索引或不适合排在联合索引列的靠前的位置，而 UUID 这类字段就比较适合做索引或排在联合索引列的靠前的位置。

因为如果索引的区分度很小，假设字段的值分布均匀，那么无论搜索哪个值都可能得到一半的数据。在这些情况下，还不如不要索引，因为 MySQL 还有一个查询优化器，查询优化器发现某个值出现在表的数据行中的百分比（惯用的百分比界线是"30%"）很高的时候，它一般会忽略索引，进行全表扫描。

联合索引 ABC，现在有个执行语句是 A = XXX and C < XXX，索引怎么走

根据最左匹配原则，A 可以走联合索引，C 不会走联合索引，但是 C 可以走索引下推

联合索引(a,b,c)，查询条件 where b > xxx and a = x 会生效吗

索引会生效，a 和 b 字段都能利用联合索引，符合联合索引最左匹配原则。

联合索引 (a, b，c)，where 条件是 a=2 and c = 1，能用到联合索引吗?

会用到联合索引，但是只有 a 才能走索引，c 无法走索引，因为不符合最左匹配原则。虽然 c 无法走索引， 但是 c 字段在 5.6 版本之后，会有索引下推的优化，能减少回表查询的次数。

索引失效有哪些?

6 种会发生索引失效的情况：

- 当我们使用左或者左右模糊匹配的时候，也就是 like %xx 或者 like %xx%这两种方式都会造成索引失效；
- 当我们在查询条件中对索引列使用函数，就会导致索引失效。
- 当我们在查询条件中对索引列进行表达式计算，也是无法走索引的。
- MySQL 在遇到字符串和数字比较的时候，会自动把字符串转为数字，然后再进行比较。如果字符串是索引列，而条件语句中的输入参数是数字的话，那么索引列会发生隐式类型转换，由于隐式类型转换是通过 CAST 函数实现的，等同于对索引列使用了函数，所以就会导致索引失效。
- 联合索引要能正确使用需要遵循最左匹配原则，也就是按照最左优先的方式进行索引的匹配，否则就会导致索引失效。
- 在 WHERE 子句中，如果在 OR 前的条件列是索引列，而在 OR 后的条件列不是索引列，那么索引会失效。

什么情况下会回表查询

从物理存储的角度来看，索引分为聚簇索引（主键索引）、二级索引（辅助索引）。

它们的主要区别如下：

- 主键索引的 B+Tree 的叶子节点存放的是实际数据，所有完整的用户记录都存放在主键索引的 B+Tree 的叶子节点里；
- 二级索引的 B+Tree 的叶子节点存放的是主键值，而不是实际数据。

所以，在查询时使用了二级索引，如果查询的数据能在二级索引里查询的到，那么就不需要回表，这个过程就是覆盖索引。

如果查询的数据不在二级索引里，就会先检索二级索引，找到对应的叶子节点，获取到主键值后，然后再检索主键索引，就能查询到数据了，这个过程就是回表。

什么是覆盖索引?

覆盖索引是指一个索引包含了查询所需的所有列，因此不需要访问表中的数据行就能完成查询。

换句话说，查询所需的所有数据都能从索引中直接获取，而不需要进行回表查询。覆盖索引能够显著提高查询性能，因为减少了访问数据页的次数，从而减少了 I/O 操作。

假设有一张表 employees，表结构如下：

```
CREATE TABLE employees (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  age INT,
  department VARCHAR(100),
  salary DECIMAL(10, 2)
);

CREATE INDEX idx_name_age_department ON employees(name, age, department);
```

如果我们有以下查询：

```
SELECT name, age, department FROM employees WHERE name = 'John';
```

在这种情况下，idx_name_age_department 是一个覆盖索引，因为它包含了查询所需的所有列：name、age 和 department。查询可以完全在索引层完成，而不需要访问表中的数据行。

如果一个列即使单列索引，又是联合索引，单独查它的话先走哪个?

mysql 优化器会分析每个索引的查询成本，然后选择成本最低的方案来执行 sql。

如果单列索引是 a，联合索引是（a ，b），那么针对下面这个查询：

```
select a, b from table where a = ? and b =?
```

优化器会选择联合索引，因为查询成本更低，查询也不需要回表，直接索引覆盖了。

索引已经建好了，那我再插入一条数据，索引会有哪些变化?

插入新数据可能导致 B+树结构的调整和索引信息的更新，以保持 B+树的平衡性和正确性，这些变化通常由数据库系统自动处理，确保数据的一致性和索引的有效性。

如果插入的数据导致叶子节点已满，可能会触发叶子节点的分裂操作，以保持 B+树的平衡性。

索引字段是不是建的越多越好?

不是，建的的越多会占用越多的空间，而且在写入频繁的场景下，对于 B+树的维护所付出的性能消耗也会越大

如果有一个字段是 status 值为 0 或者 1，适合建索引吗

不适合，区分度低的字段不适合建立索引。

索引的优缺点?

索引最大的好处是提高查询速度，但是索引也是有缺点的，比如：

- 需要占用物理空间，数量越大，占用空间越大；
- 创建索引和维护索引要耗费时间，这种时间随着数据量的增加而增大；
- 会降低表的增删改的效率，因为每次增删改索引，B+ 树为了维护索引有序性，都需要进行动态维护。

所以，索引不是万能钥匙，它也是根据场景来使用的。

怎么决定建立哪些索引?

> 什么时候适用索引？

- 字段有唯一性限制的，比如商品编码；
- 经常用于 `WHERE` 查询条件的字段，这样能够提高整个表的查询速度，如果查询条件不是一个字段，可以建立联合索引。

- 经常用于 `GROUP BY` 和 `ORDER BY` 的字段，这样在查询的时候就不需要再去做一次排序了，因为我们都已经知道了建立索引之后在 B+Tree 中的记录都是排序好的。

> 什么时候不需要创建索引？

- `WHERE` 条件，`GROUP BY`，`ORDER BY` 里用不到的字段，索引的价值是快速定位，如果起不到定位的字段通常是不需要创建索引的，因为索引是会占用物理空间的。
- 字段中存在大量重复数据，不需要创建索引，比如性别字段，只有男女，如果数据库表中，男女的记录分布均匀，那么无论搜索哪个值都可能得到一半的数据。在这些情况下，还不如不要索引，因为 MySQL 还有一个查询优化器，查询优化器发现某个值出现在表的数据行中的百分比很高的时候，它一般会忽略索引，进行全表扫描。
- 表数据太少的时候，不需要创建索引；
- 经常更新的字段不用创建索引，比如不要对电商项目的用户余额建立索引，因为索引字段频繁修改，由

索引优化详细讲讲

常见优化索引的方法：

- 前缀索引优化：使用前缀索引是为了减小索引字段大小，可以增加一个索引页中存储的索引值，有效提高索引的查询速度。在一些大字符串的字段作为索引时，使用前缀索引可以帮助我们减小索引项的大小。
- 覆盖索引优化：覆盖索引是指 SQL 中 query 的所有字段，在索引 B+Tree 的叶子节点上都能找得到的那些索引，从二级索引中查询得到记录，而不需要通过聚簇索引查询获得，可以避免回表的操作。
- 主键索引最好是自增的：
    - 如果我们使用自增主键，那么每次插入的新数据就会按顺序添加到当前索引节点的位置，不需要移动已有的数据，当页面写满，就会自动开辟一个新页面。因为每次**插入一条新记录，都是追加操作，不需要重新移动数据**，因此这种插入数据的方法效率非常高。
    - 如果我们使用非自增主键，由于每次插入主键的索引值都是随机的，因此每次插入新的数据时，就可能会插入到现有数据页中间的某个位置，这将不得不移动其它数据来满足新数据的插入，甚至需要从一个页面复制数据到另外一个页面，我们通常将这种情况称为**页分裂**。页分裂还有可能会造成大量的内存碎片，导致索引结构不紧凑，从而影响查询效率。

- 防止索引失效：
    - 当我们使用左或者左右模糊匹配的时候，也就是 `like %xx` 或者 `like %xx%`这两种方式都会造成索引失效；
    - 当我们在查询条件中对索引列做了计算、函数、类型转换操作，这些情况下都会造成索引失效；
    - 联合索引要能正确使用需要遵循最左匹配原则，也就是按照最左优先的方式进行索引的匹配，否则就会导致索引失效。
    - 在 WHERE 子句中，如果在 OR 前的条件列是索引列，而在 OR 后的条件列不是索引列，那么索引会失效。

了解过前缀索引吗?

使用前缀索引是为了减小索引字段大小，可以增加一个索引页中存储的索引值，有效提高索引的查询速度。在一些大字符串的字段作为索引时，使用前缀索引可以帮助我们减小索引项的大小。

事务

事务的特性是什么？如何实现的?

- **原子性（Atomicity）**：一个事务中的所有操作，要么全部完成，要么全部不完成，不会结束在中间某个环节，而且事务在执行过程中发生错误，会被回滚到事务开始前的状态，就像这个事务从来没有执行过一样，就好比买一件商品，购买成功时，则给商家付了钱，商品到手；购买失败时，则商品在商家手中，消费者的钱也没花出去。
- **一致性（Consistency）**：是指事务操作前和操作后，数据满足完整性约束，数据库保持一致性状态。比如，用户 A 和用户 B 在银行分别有 800 元和 600 元，总共 1400 元，用户 A 给用户 B 转账 200 元，分为两个步骤，从 A 的账户扣除 200 元和对 B 的账户增加 200 元。一致性就是要求上述步骤操作后，最后的结果是用户 A 还有 600 元，用户 B 有 800 元，总共 1400 元，而不会出现用户 A 扣除了 200 元，但用户 B 未增加的情况（该情况，用户 A 和 B 均为 600 元，总共 1200 元）。
- **隔离性（Isolation）**：数据库允许多个并发事务同时对其数据进行读写和修改的能力，隔离性可以防止多个事务并发执行时由于交叉执行而导致数据的不一致，因为多个事务同时使用相同的数据时，不会相互干扰，每个事务都有一个完整的数据空间，对其他并发事务是隔离的。也就是说，消费者购买商品这个事务，是不影响其他消费者购买的。

- **持久性（Durability）**：事务处理结束后，对数据的修改就是永久的，即便系统故障也不会丢失。

MySQL InnoDB 引擎通过什么技术来保证事务的这四个特性的呢？

- 持久性是通过 redo log （重做日志）来保证的；
- 原子性是通过 undo log（回滚日志） 来保证的；
- 隔离性是通过 MVCC（多版本并发控制） 或锁机制来保证的；
- 一致性则是通过持久性+原子性+隔离性来保证；

mysql 可能出现什么和并发相关问题?

MySQL 服务端是允许多个客户端连接的，这意味着 MySQL 会出现同时处理多个事务的情况。

那么**在同时处理多个事务的时候，就可能出现脏读（dirty read）、不可重复读（non-repeatable read）、幻读（phantom read）的问题**。

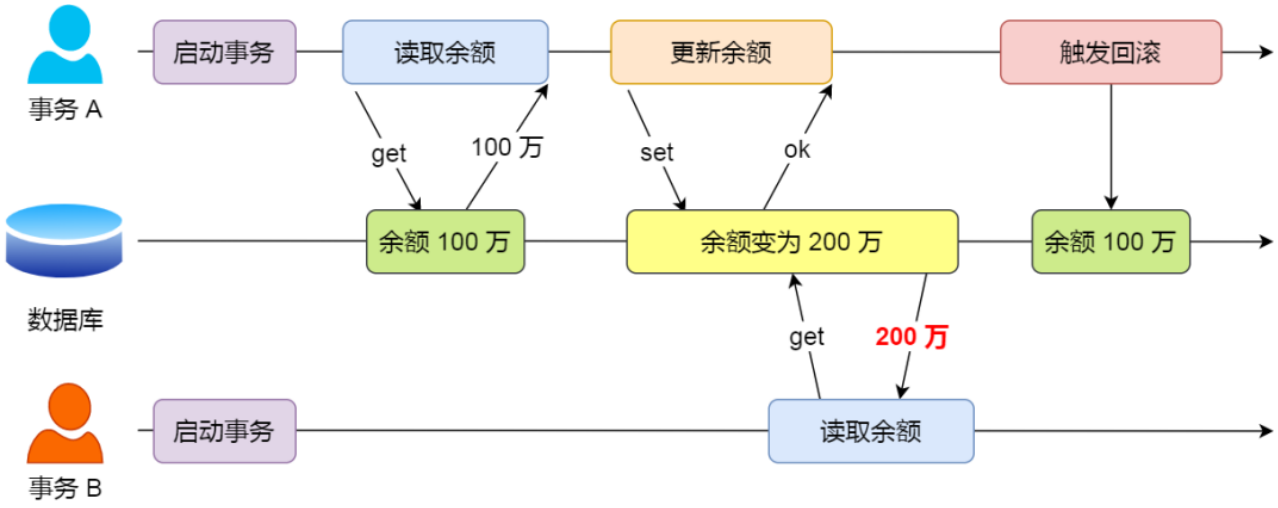
接下来，通过举例子给大家说明，这些问题是如何发生的。

> 脏读

如果一个事务「读到」了另一个「未提交事务修改过的数据」，就意味着发生了「脏读」现象。

举个栗子。

假设有 A 和 B 这两个事务同时在处理，事务 A 先开始从数据库中读取小林的余额数据，然后再执行更新操作，如果此时事务 A 还没有提交事务，而此时正好事务 B 也从数据库中读取小林的余额数据，那么事务 B 读取到的余额数据是刚才事务 A 更新后的数据，即使没有提交事务。



因为事务 A 是还没提交事务的，也就是它随时可能发生回滚操作，**如果在上面这种情况事务 A 发生了回滚，那么事务 B 刚才得到的数据就是过期的数据，这种现象就被称为脏读。**

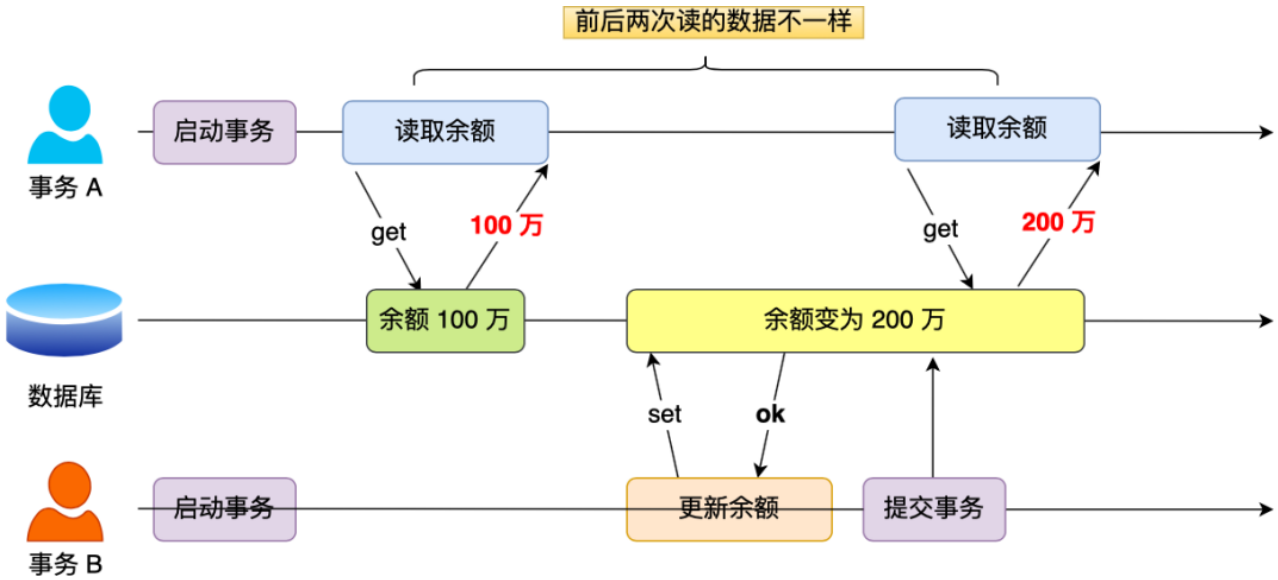
> 不可重复读

在一个事务内多次读取同一个数据，如果出现前后两次读到的数据不一样的情况，就意味着发生了「不可重复读」现象。

举个栗子。

假设有 A 和 B 这两个事务同时在处理，事务 A 先开始从数据库中读取小林的余额数据，然后继续执行代码逻辑处理，**在这过程中如果事务 B 更新了这条数据，并提交了事务，那么当事务 A 再次读取该数据时，就会发现前后两次读到的数据是不一致的，这种现象就被称为不可重复读。

**

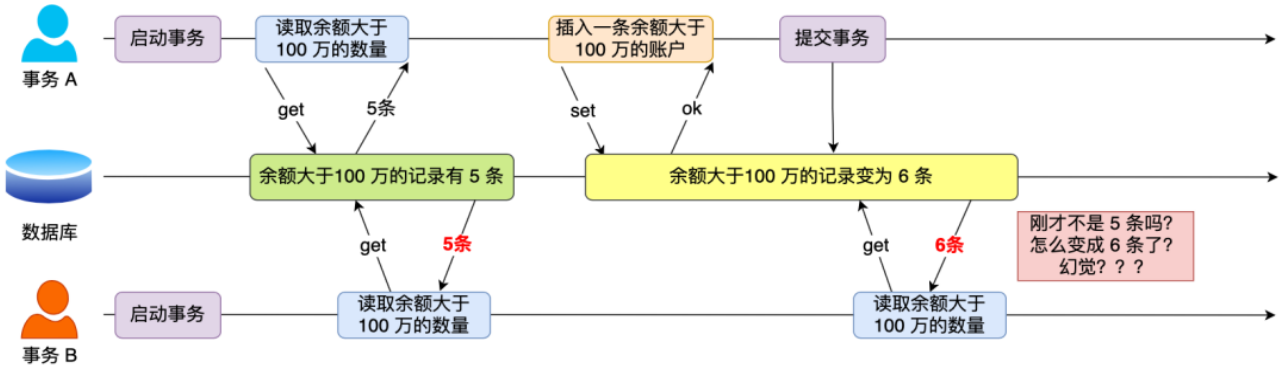


> 幻读

在一个事务内多次查询某个符合查询条件的「记录数量」，如果出现前后两次查询到的记录数量不一样的情况，就意味着发生了「幻读」现象。

举个栗子。

假设有 A 和 B 这两个事务同时在处理，事务 A 先开始从数据库查询账户余额大于 100 万的记录，发现共有 5 条，然后事务 B 也按相同的搜索条件也是查询出了 5 条记录。



接下来，事务 A 插入了一条余额超过 100 万的账号，并提交了事务，此时数据库超过 100 万余额的账号个数就变为 6。

然后事务 B 再次查询账户余额大于 100 万的记录，此时查询到的记录数量有 6 条，**发现和前一次读到的记录数量不一样了，就感觉发生了幻觉一样，这种现象就被称为幻读。**

哪些场景不适合脏读，举个例子?

脏读是指一个事务在读取到另一个事务未提交的数据时发生。脏读可能会导致不一致的数据被读取，并可能引起问题。以下是一些不适合脏读的场景：

- **银行系统**：在银行系统中，如果一个账户的余额正在被调整但尚未提交，另一个事务读取了这个临时的余额，可能会导致客户看到不正确的余额。
- **库存管理系统**：在一个库存管理系统中，如果一个商品的数量正在被更新但尚未提交，另一个事务读取了这个临时的数量，可能会导致库存管理错误。
- **在线订单系统**：在一个在线订单系统中，如果一个订单正在被修改但尚未提交，另一个事务读取了这个临时的订单状态，可能导致订单状态显示错误，客户收到不准确的信息。

在以上这些场景中，脏读可能导致严重的问题，因此应该避免发生脏读，保证数据的一致性和准确性。

mysql 的是怎么解决并发问题的?

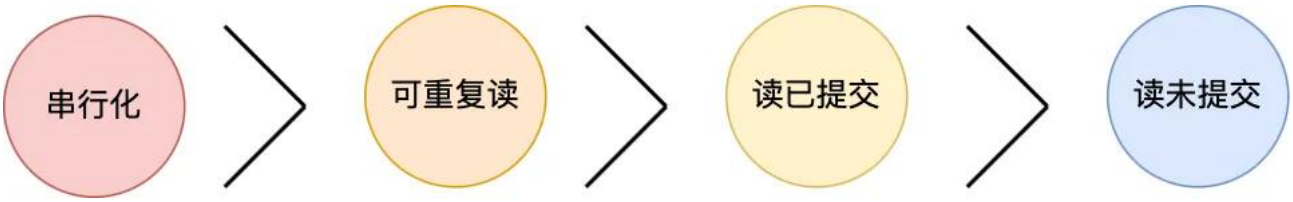
- 锁机制：Mysql 提供了多种锁机制来保证数据的一致性，包括行级锁、表级锁、页级锁等。通过锁机制，可以在读写操作时对数据进行加锁，确保同时只有一个操作能够访问或修改数据。
- 事务隔离级别：Mysql 提供了多种事务隔离级别，包括读未提交、读已提交、可重复读和串行化。通过设置合适的事务隔离级别，可以在多个事务并发执行时，控制事务之间的隔离程度，以避免数据不一致的问题。

- MVCC（多版本并发控制）：Mysql 使用 MVCC 来管理并发访问，它通过在数据库中保存不同版本的数据来实现不同事务之间的隔离。在读取数据时，Mysql 会根据事务的隔离级别来选择合适的数据版本，从而保证数据的一致性。

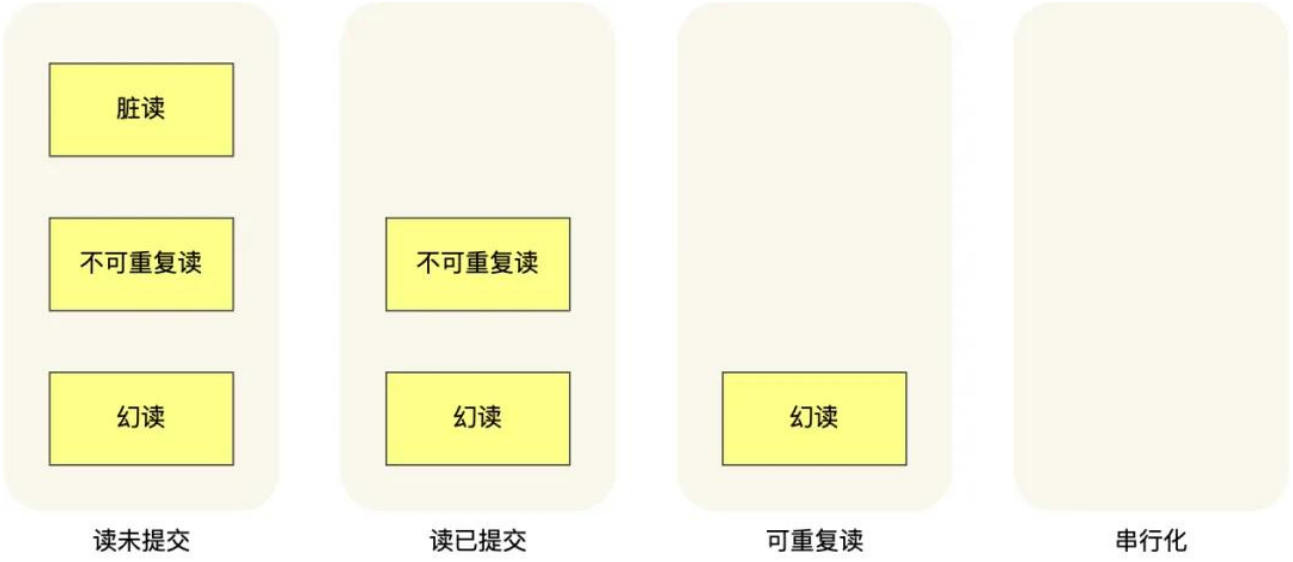
事务的隔离级别有哪些?

- **读未提交**（read uncommitted），指一个事务还没提交时，它做的变更就能被其他事务看到；
- **读提交**（read committed），指一个事务提交之后，它做的变更才能被其他事务看到；
- **可重复读**（repeatable read），指一个事务执行过程中看到的数据，一直跟这个事务启动时看到的数据是一致的，**MySQL InnoDB 引擎的默认隔离级别**；
- **串行化**（serializable）；会对记录加上读写锁，在多个事务对这条记录进行读写操作时，如果发生了读写冲突的时候，后访问的事务必须等前一个事务执行完成，才能继续执行；

按隔离水平高低排序如下：



针对不同的隔离级别，并发事务时可能发生的现象也会不同。



也就是说：

- 在「读未提交」隔离级别下，可能发生脏读、不可重复读和幻读现象；
- 在「读提交」隔离级别下，可能发生不可重复读和幻读现象，但是不可能发生脏读现象；
- 在「可重复读」隔离级别下，可能发生幻读现象，但是不可能脏读和不可重复读现象；

- 在「串行化」隔离级别下，脏读、不可重复读和幻读现象都不可能会发生。

接下来，举个具体的例子来说明这四种隔离级别，有一张账户余额表，里面有一条账户余额为 100 万的记录。然后有两个并发的事务，事务 A 只负责查询余额，事务 B 则会将我的余额改成 200 万，下面是按照时间顺序执行两个事务的行为：

启动事务 A	启动事务 B
查得得到余额 100 万	
	查得得到余额 100 万
	将余额 100 万改成 200 万
查得得到余额 V1	
	提交事务 B
查得得到余额 V2	
提交事务 A	
查得得到余额 V3	

在不同隔离级别下，事务 A 执行过程中查询到的余额可能会不同：

- 在「读未提交」隔离级别下，事务 B 修改余额后，虽然没有提交事务，但是此时的余额已经可以被事务 A 看见了，于是事务 A 中余额 V1 查询的值是 200 万，余额 V2、V3 自然也是 200 万了；
- 在「读提交」隔离级别下，事务 B 修改余额后，因为没有提交事务，所以事务 A 中余额 V1 的值还是 100 万，等事务 B 提交完后，最新的余额数据才能被事务 A 看见，因此额 V2、V3 都是 200 万；
- 在「可重复读」隔离级别下，事务 A 只能看见启动事务时的数据，所以余额 V1、余额 V2 的值都是 100 万，当事务 A 提交事务后，就能看见最新的余额数据了，所以余额 V3 的值是 200 万；

- 在「串行化」隔离级别下，事务 B 在执行将余额 100 万修改为 200 万时，由于此前事务 A 执行了读操作，这样就发生了读写冲突，于是就会被锁住，直到事务 A 提交后，事务 B 才可以继续执行，所以从 A 的角度看，余额 V1、V2 的值是 100 万，余额 V3 的值是 200 万。

这四种隔离级别具体是如何实现的呢？

- 对于「读未提交」隔离级别的事务来说，因为可以读到未提交事务修改的数据，所以直接读取最新的数据就好了；
- 对于「串行化」隔离级别的事务来说，通过加读写锁的方式来避免并行访问；
- 对于「读提交」和「可重复读」隔离级别的事务来说，它们是通过 Read View 来实现的，它们的区别在于创建 Read View 的时机不同，**「读提交」隔离级别是在「每个语句执行前」都会重新生成一个 Read View，而「可重复读」隔离级别是「启动事务时」生成一个 Read View，然后整个事务期间都在用这个 Read View**。

mysql 默认级别是什么?

可重复读隔离级别

可重复读隔离级别下，A 事务提交的数据，在 B 事务能看见吗?

可重复读隔离级是由 MVCC（多版本并发控制）实现的，实现的方式是开始事务后（执行 begin 语句后），在执行第一个查询语句后，会创建一个 Read View，**后续的查询语句利用这个 Read View，通过这个 Read View 就可以在 undo log 版本链找到事务开始时的数据，所以事务过程中每次查询的数据都是一样的**，即使中途有其他事务插入了新纪录，是查询不出来这条数据的。

举个例子说可重复读下的幻读问题

可重复读隔离级别下虽然很大程度上避免了幻读，但是还是没有能完全解决幻读。

我举例一个可重复读隔离级别发生幻读现象的场景。以这张表作为例子：

id	name	score
1	小林	50
2	小明	60
3	小红	70
4	小蓝	80

事务 A 执行查询 id = 5 的记录，此时表中是没有该记录的，所以查询不出来。

```
# 事务 A
```

```
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from t_stu where id = 5;
Empty set (0.01 sec)
```

然后事务 B 插入一条 id = 5 的记录，并且提交了事务。

```
# 事务 B
mysql> begin;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into t_stu values(5, '小美', 18);
Query OK, 1 row affected (0.00 sec)

mysql> commit;
Query OK, 0 rows affected (0.00 sec)
```

此时，**事务 A 更新 id = 5 这条记录，对没错，事务 A 看不到 id = 5 这条记录，但是他去更新了这条记录，这场景确实很违和，然后再次查询 id = 5 的记录，事务 A 就能看到事务 B 插入的纪录了，幻读就是发生在这种违和的场景**。

```
# 事务 A
mysql> update t_stu set name = '小林 coding' where id = 5;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from t_stu where id = 5;
+----+--------------+------+
| id | name         | age  |
+----+--------------+------+
|  5 | 小林 coding   |   18 |
+----+--------------+------+
1 row in set (0.00 sec)
```

整个发生幻读的时序图如下：

在可重复读隔离级别下，事务 A 第一次执行普通的 select 语句时生成了一个 ReadView，之后事务 B 向表中新插入了一条 id = 5 的记录并提交。接着，事务 A 对 id = 5 这条记录进行了更新操作，在这个时刻，这条新记录的 trx_id 隐藏列的值就变成了事务 A 的事务 id，之后事务 A 再使用普通 select 语句去查询这条记录时就可以看到这条记录了，于是就发生了幻读。

因为这种特殊现象的存在，所以我们认为 **MySQL Innodb 中的 MVCC 并不能完全避免幻读现象**。

Mysql 设置了可重读隔离级后，怎么保证不发生幻读?

尽量在开启事务之后，马上执行 select ... for update 这类锁定读的语句，因为它会对记录加 next-key lock，从而避免其他事务插入一条新记录，就避免了幻读的问题。

串行化隔离级别是通过什么实现的?

是通过行级锁来实现的，序列化隔离级别下，普通的 select 查询是会对记录加 S 型的 next-key 锁，其他事务就没没办法对这些已经加锁的记录进行增删改操作了，从而避免了脏读、不可重复读和幻读现象。

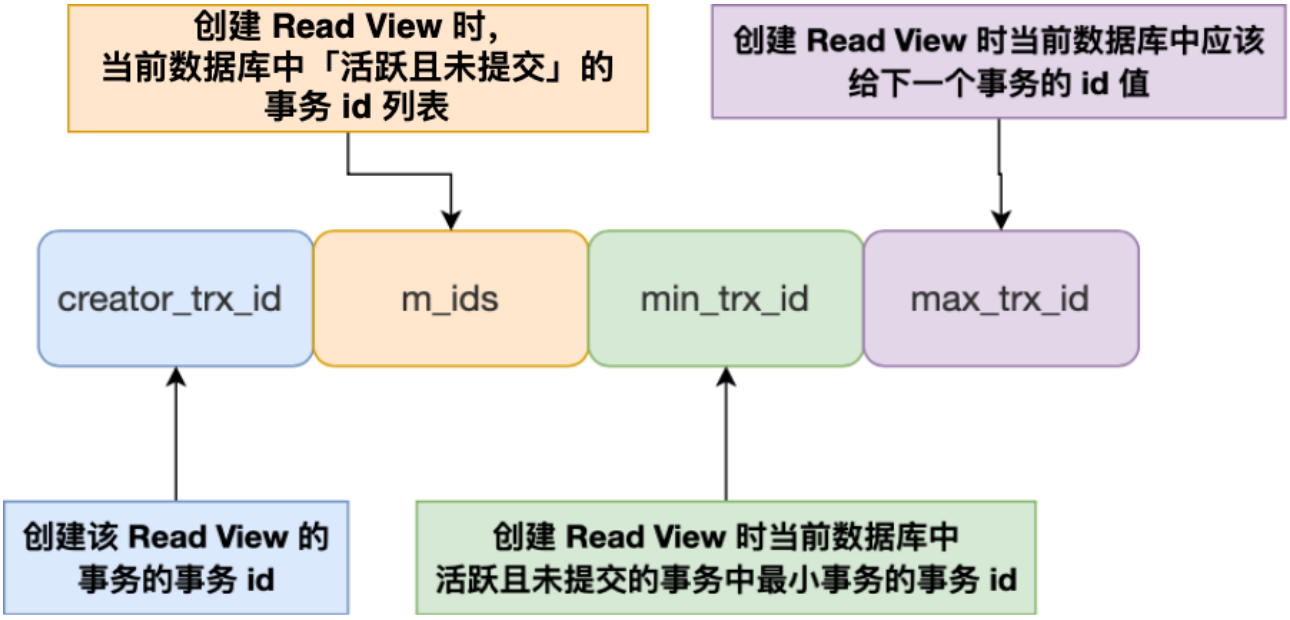
介绍 MVCC 实现原理

MVCC 允许多个事务同时读取同一行数据，而不会彼此阻塞，每个事务看到的数据版本是该事务开始时的数据版本。这意味着，如果其他事务在此期间修改了数据，正在运行的事务仍然看到的是它开始时的数据状态，从而实现了非阻塞读操作。

对于「读提交」和「可重复读」隔离级别的事务来说，它们是通过 Read View 来实现的，它们的区别在于创建 Read View 的时机不同，大家可以把 Read View 理解成一个数据快照，就像相机拍照那样，定格某一时刻的风景。

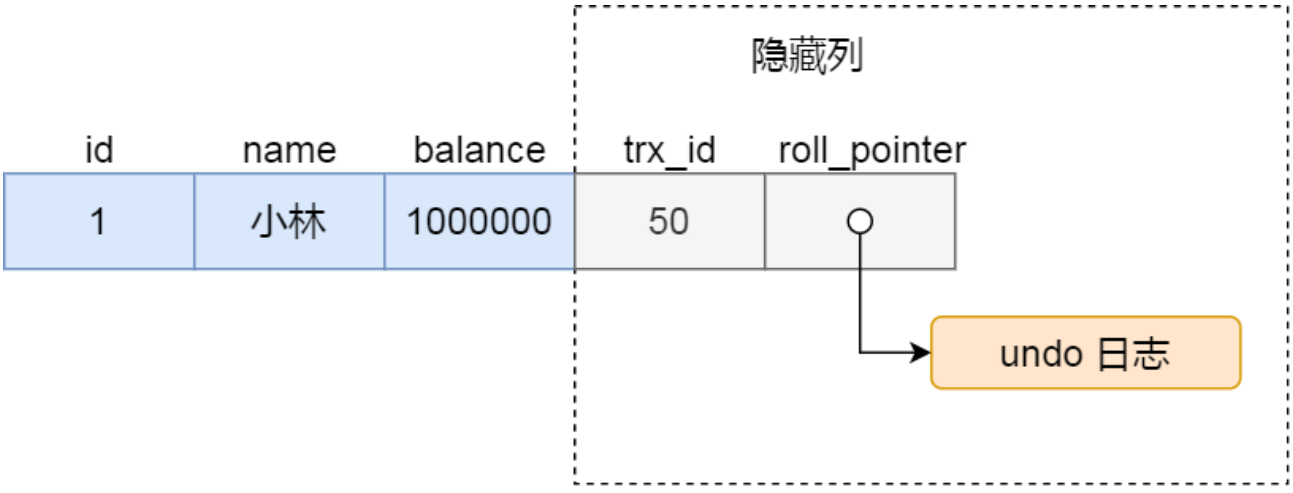
- 「读提交」隔离级别是在「每个 select 语句执行前」都会重新生成一个 Read View；
- 「可重复读」隔离级别是执行第一条 select 时，生成一个 Read View，然后整个事务期间都在用这个 Read View。

Read View 有四个重要的字段：



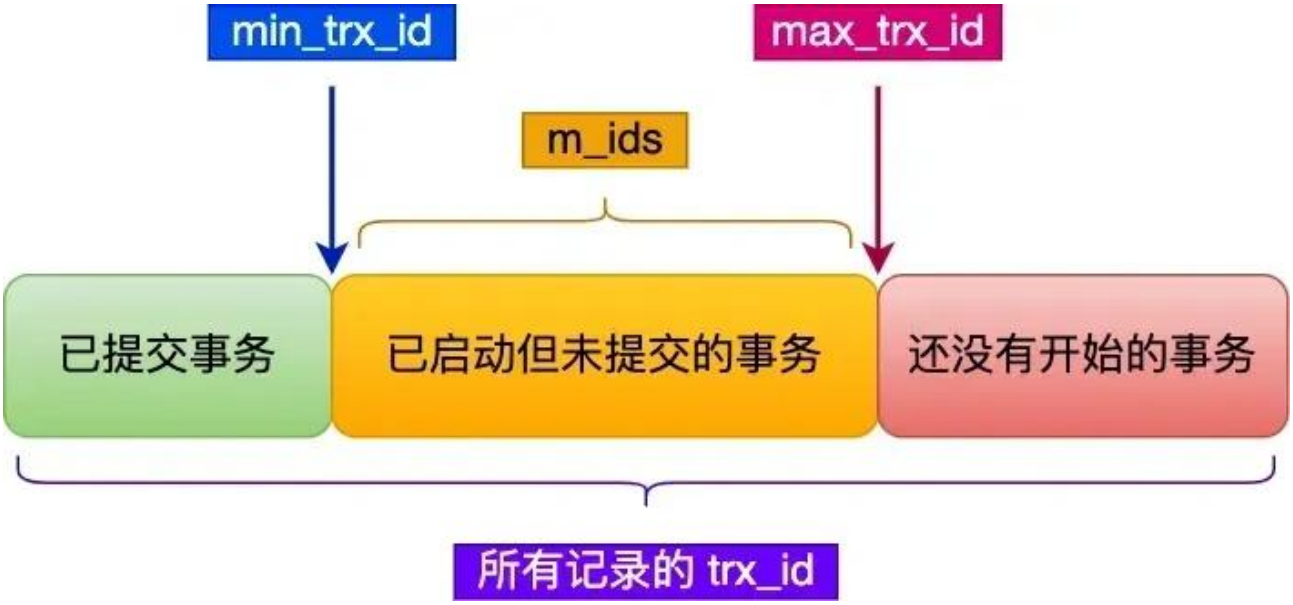
- m_ids ：指的是在创建 Read View 时，当前数据库中「活跃事务」的**事务 id 列表**，注意是一个列表，**“活跃事务”指的就是，启动了但还没提交的事务**。
- min_trx_id ：指的是在创建 Read View 时，当前数据库中「活跃事务」中事务 **id 最小的事务**，也就是 m_ids 的最小值。
- max_trx_id ：这个并不是 m_ids 的最大值，而是**创建 Read View 时当前数据库中应该给下一个事务的 id 值**，也就是全局事务中最大的事务 id 值 + 1；
- creator_trx_id ：指的是**创建该 Read View 的事务的事务 id**。

对于使用 InnoDB 存储引擎的数据库表，它的聚簇索引记录中都包含下面两个隐藏列：



- trx_id，当一个事务对某条聚簇索引记录进行改动时，就会**把该事务的事务 id 记录在 trx_id 隐藏列里**；
- roll_pointer，每次对某条聚簇索引记录进行改动时，都会把旧版本的记录写入到 undo 日志中，然后**这个隐藏列是个指针，指向每一个旧版本记录**，于是就可以通过它找到修改前的记录。

在创建 Read View 后，我们可以将记录中的 trx_id 划分这三种情况：



一个事务去访问记录的时候，除了自己的更新记录总是可见之外，还有这几种情况：

- 如果记录的 trx_id 值小于 Read View 中的 min_trx_id 值，表示这个版本的记录是在创建 Read View **前**已经提交的事务生成的，所以该版本的记录对当前事务**可见**。
- 如果记录的 trx_id 值大于等于 Read View 中的 max_trx_id 值，表示这个版本的记录是在创建 Read View **后**才启动的事务生成的，所以该版本的记录对当前事务**不可见**。
- 如果记录的 trx_id 值在 Read View 的 min_trx_id 和 max_trx_id 之间，需要判断 trx_id 是否在 m_ids 列表中：
- 如果记录的 trx_id **在** m_ids 列表中，表示生成该版本记录的活跃事务依然活跃着（还没提交事务），所以该版本的记录对当前事务**不可见**。
- 如果记录的 trx_id **不在** m_ids 列表中，表示生成该版本记录的活跃事务已经被提交，所以该版本的记录对当前事务**可见**。

这种通过「版本链」来控制并发事务访问同一个记录时的行为就叫 MVCC（多版本并发控制）。

一条 update 是不是原子性的？为什么？

是原子性，主要通过锁+undolog 日志保证原子性的

- 执行 update 的时候，会加行级别锁，保证了一个事务更新一条记录的时候，不会被其他事务干扰。
- 事务执行过程中，会生成 undolog，如果事务执行失败，就可以通过 undolog 日志进行回滚。

滥用事务，或者一个事务里有特别多 sql 的弊端?

事务的资源在事务提交之后才会释放的，比如存储资源、锁。

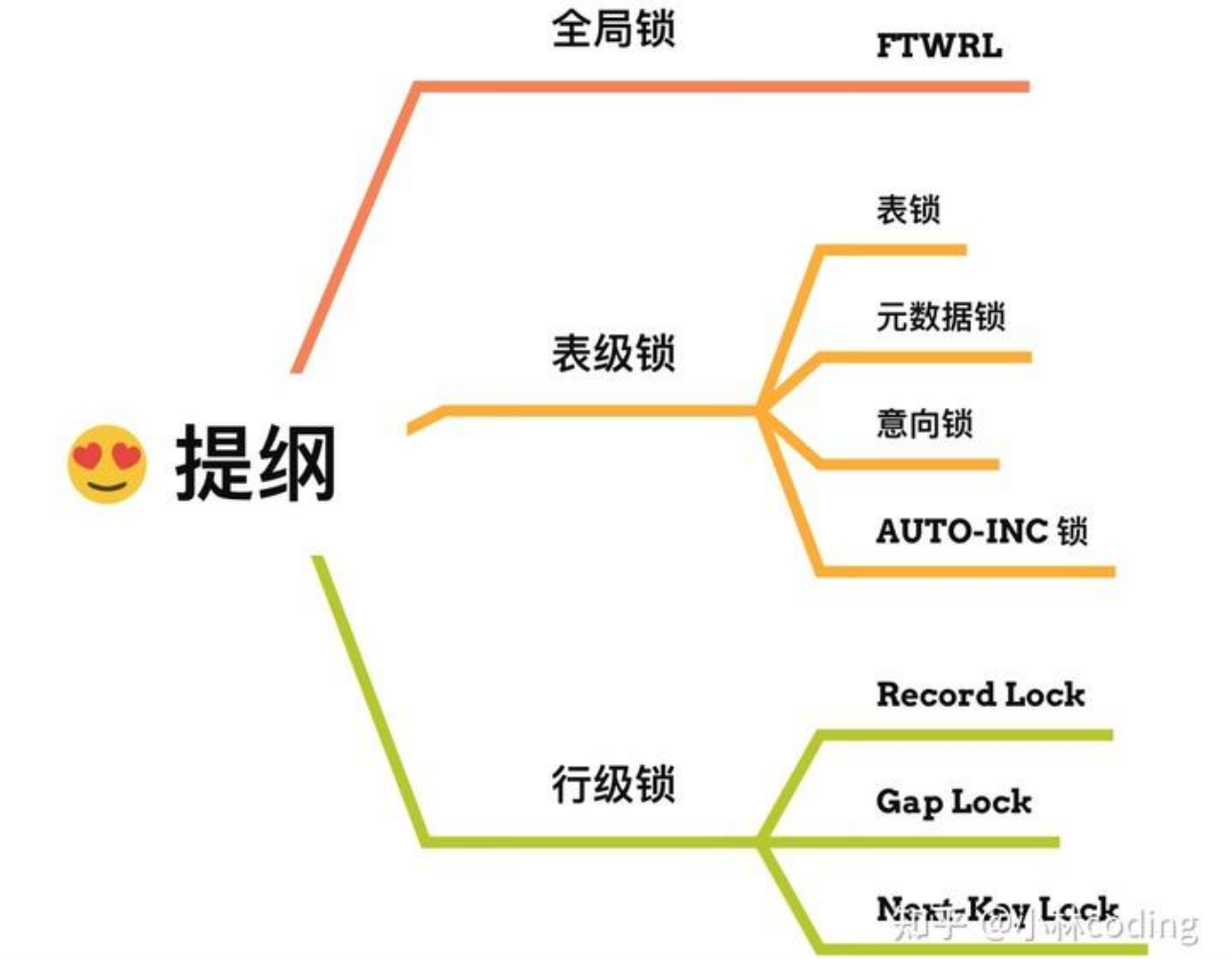
如果一个事务特别多 sql，那么会带来这些问题：

- 如果一个事务特别多 sql，锁定的数据太多，容易造成大量的死锁和锁超时。
- 回滚记录会占用大量存储空间，事务回滚时间长。在 MySQL (opens new window)中，实际上每条记录在更新的时候都会同时记录一条回滚操作。记录上的最新值，通过回滚操作，都可以得到前一个状态的值，sql 越多，所需要保存的回滚数据就越多。
- 执行时间长，容易造成主从延迟，主库上必须等事务执行完成才会写入 binlog，再传给备库。所以，如果一个主库上的语句执行 10 分钟，那这个事务很可能就会导致从库延迟 10 分钟

锁

讲一下 mysql 里有哪些锁?

在 MySQL 里，根据加锁的范围，可以分为**全局锁、表级锁和行锁**三类。



- **全局锁**：通过 flush tables with read lock 语句会将整个数据库就处于只读状态了，这时其他线程执行以下操作，增删改或者表结构修改都会阻塞。全局锁主要应用于做**全库逻辑备份**，这样在备份数据库期间，不会因为数据或表结构的更新，而出现备份文件的数据与预期的不一样。
- **表级锁**：MySQL 里面表级别的锁有这几种：
    - 表锁：通过 lock tables 语句可以对表加表锁，表锁除了会限制别的线程的读写外，也会限制本线程接下来的读写操作。
    - 元数据锁：当我们对数据库表进行操作时，会自动给这个表加上 MDL，对一张表进行 CRUD 操作时，加的是 **MDL 读锁**；对一张表做结构变更操作的时候，加的是 **MDL 写锁**；MDL 是为了保证当用户对表执行 CRUD 操作时，防止其他线程对这个表结构做了变更。
    - 意向锁：当执行插入、更新、删除操作，需要先对表加上「意向独占锁」，然后对该记录加独占锁。**意向锁的目的是为了快速判断表里是否有记录被加锁**。

- **行级锁**：InnoDB 引擎是支持行级锁的，而 MyISAM 引擎并不支持行级锁。
- 记录锁，锁住的是一条记录。而且记录锁是有 S 锁和 X 锁之分的，满足读写互斥，写写互斥
- 间隙锁，只存在于可重复读隔离级别，目的是为了解决可重复读隔离级别下幻读的现象。
- Next-Key Lock 称为临键锁，是 Record Lock + Gap Lock 的组合，锁定一个范围，并且锁定记录本身。

数据库的表锁和行锁有什么作用?

表锁的作用：

- **整体控制**：表锁可以用来控制整个表的并发访问，当一个事务获取了表锁时，其他事务无法对该表进行任何读写操作，从而确保数据的完整性和一致性。
- **粒度大**：表锁的粒度比较大，在锁定表的情况下，可能会影响到整个表的其他操作，可能会引起锁竞争和性能问题。
- **适用于大批量操作**：表锁适合于需要大批量操作表中数据的场景，例如表的重建、大量数据的加载等。

行锁的作用：

- **细粒度控制**：行锁可以精确控制对表中某行数据的访问，使得其他事务可以同时访问表中的其他行数据，在并发量大的系统中能够提高并发性能。
- **减少锁冲突**：行锁不会像表锁那样造成整个表的锁冲突，减少了锁竞争的可能性，提高了并发访问的效率。
- **适用于频繁单行操作**：行锁适合于需要频繁对表中单独行进行操作的场景，例如订单系统中的订单修改、删除等操作。

MySQL 两个线程的 update 语句同时处理一条数据，会不会有阻塞?

如果是两个事务同时更新了 id = 1，比如 update ... where id = 1，那么是会阻塞的。因为 InnoDB 存储引擎实现了行级锁。

当 A 事务对 id =1 这行记录进行更新时，会对主键 id 为 1 的记录加 X 类型的记录锁，这样第二事务对 id = 1 进行更新时，发现已经有记录锁了，就会陷入阻塞状态。

两条 update 语句处理一张表的不同的主键范围的记录，一个<10，一个>15，会不会遇到阻塞？底层是为什么的？

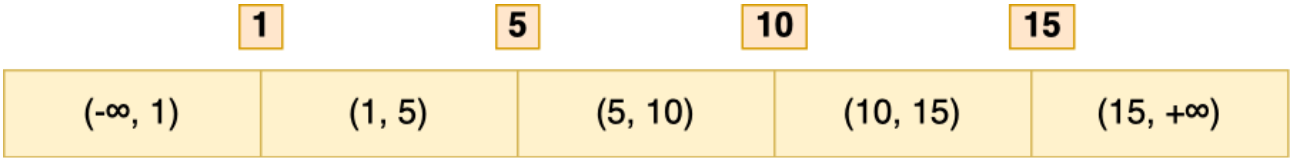
不会，因为锁住的范围不一样，不会形成冲突。

- 第一条 update sql 的话（ id<10），锁住的范围是（-∞，10）
- 第二条 update sql 的话（id >15），锁住的范围是（15，+∞）

如果 2 个范围不是主键或索引？还会阻塞吗？

如果 2 个范围查询的字段不是索引的话，那就代表 update 没有用到索引，这时候触发了全表扫描，全部索引都会加行级锁，这时候第二条 update 执行的时候，就会阻塞了。

因为如果 update 没有用到索引，在扫描过程中会对索引加锁，所以全表扫描的场景下，所有记录都会被加锁，也就是这条 update 语句产生了 4 个记录锁和 5 个间隙锁，相当于锁住了全表。



日志

日志文件是分成了哪几种？

- redo log 重做日志，是 Innodb 存储引擎层生成的日志，实现了事务中的**持久性**，主要**用于掉电等故障恢复**；
- undo log 回滚日志，是 Innodb 存储引擎层生成的日志，实现了事务中的**原子性**，主要**用于事务回滚和 MVCC**。
- bin log 二进制日志，是 Server 层生成的日志，主要**用于数据备份和主从复制**；
- relay log 中继日志，用于主从复制场景下，slave 通过 io 线程拷贝 master 的 bin log 后本地生成的日志
- 慢查询日志，用于记录执行时间过长的 sql，需要设置阈值后手动开启

讲一下 binlog

MySQL 在完成一条更新操作后，Server 层还会生成一条 binlog，等之后事务提交的时候，会将该事物执行过程中产生的所有 binlog 统一写 入 binlog 文件，binlog 是 MySQL 的 Server 层实现的日志，所有存储引擎都可以使用。

binlog 是追加写，写满一个文件，就创建一个新的文件继续写，不会覆盖以前的日志，保存的是全量的日志，用于备份恢复、主从复制；

binlog 文件是记录了所有数据库表结构变更和表数据修改的日志，不会记录查询类的操作，比如 SELECT 和 SHOW 操作。

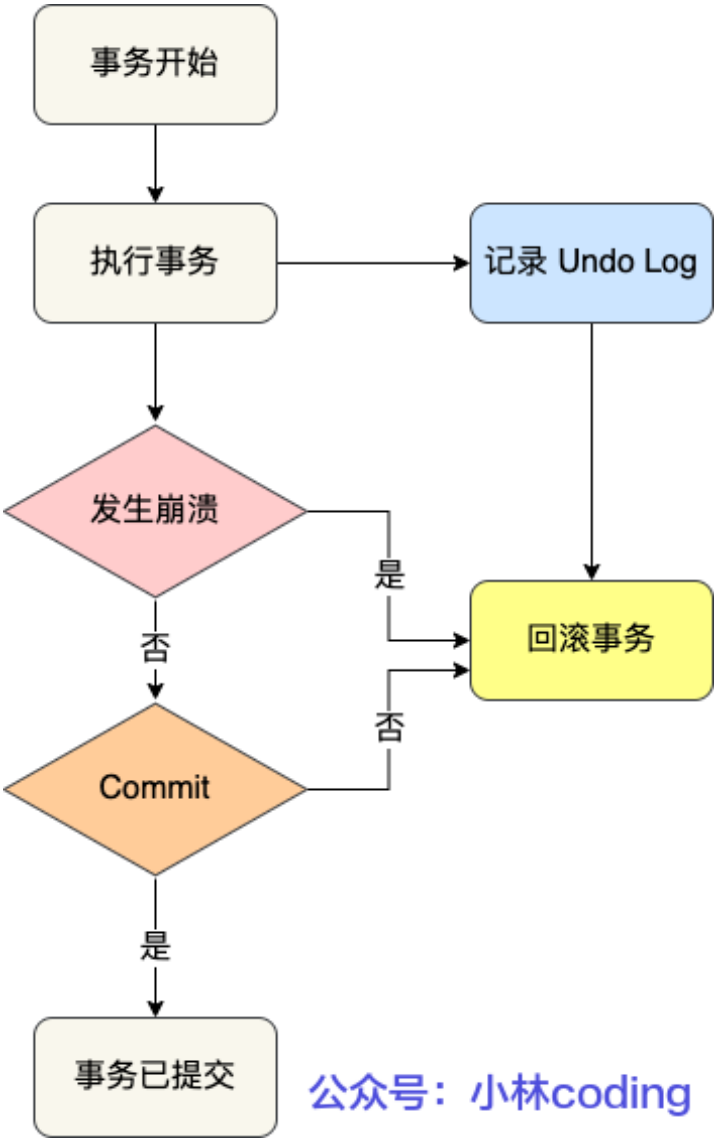
binlog 有 3 种格式类型，分别是 STATEMENT（默认格式）、ROW、 MIXED，区别如下：

- STATEMENT：每一条修改数据的 SQL 都会被记录到 binlog 中（相当于记录了逻辑操作，所以针对这种格式， binlog 可以称为逻辑日志），主从复制中 slave 端再根据 SQL 语句重现。但 STATEMENT 有动态函数的问题，比如你用了 uuid 或者 now 这些函数，你在主库上执行的结果并不是你在从库执行的结果，这种随时在变的函数会导致复制的数据不一致；
- ROW：记录行数据最终被修改成什么样了（这种格式的日志，就不能称为逻辑日志了），不会出现 STATEMENT 下动态函数的问题。但 ROW 的缺点是每行数据的变化结果都会被记录，比如执行批量 update 语句，更新多少行数据就会产生多少条记录，使 binlog 文件过大，而在 STATEMENT 格式下只会记录一个 update 语句而已；
- MIXED：包含了 STATEMENT 和 ROW 模式，它会根据不同的情况自动使用 ROW 模式和 STATEMENT 模式；

UndoLog 日志的作用是什么?

undo log 是一种用于撤销回退的日志，**它保证了事务的 ACID 特性中的原子性**（Atomicity）。

在事务没提交之前，MySQL 会先记录更新前的数据到 undo log 日志文件里面，当事务回滚时，可以利用 undo log 来进行回滚。如下图：



每当 InnoDB 引擎对一条记录进行操作（修改、删除、新增）时，要把回滚时需要的信息都记录到 undo log 里，比如：

- 在**插入**一条记录时，要把这条记录的主键值记下来，这样之后回滚时只需要把这个主键值对应的记录**删掉**就好了；
- 在**删除**一条记录时，要把这条记录中的内容都记下来，这样之后回滚时再把由这些内容组成的记录**插入**到表中就好了；
- 在**更新**一条记录时，要把被更新的列的旧值记下来，这样之后回滚时再把这些列**更新为旧值**就好了。

在发生回滚时，就读取 undo log 里的数据，然后做原先相反操作。比如当 delete 一条记录时，undo log 中会把记录中的内容都记下来，然后执行回滚操作的时候，就读取 undo log 里的数据，然后进行 insert 操作。

有了 undolog 为啥还需要 redolog 呢?

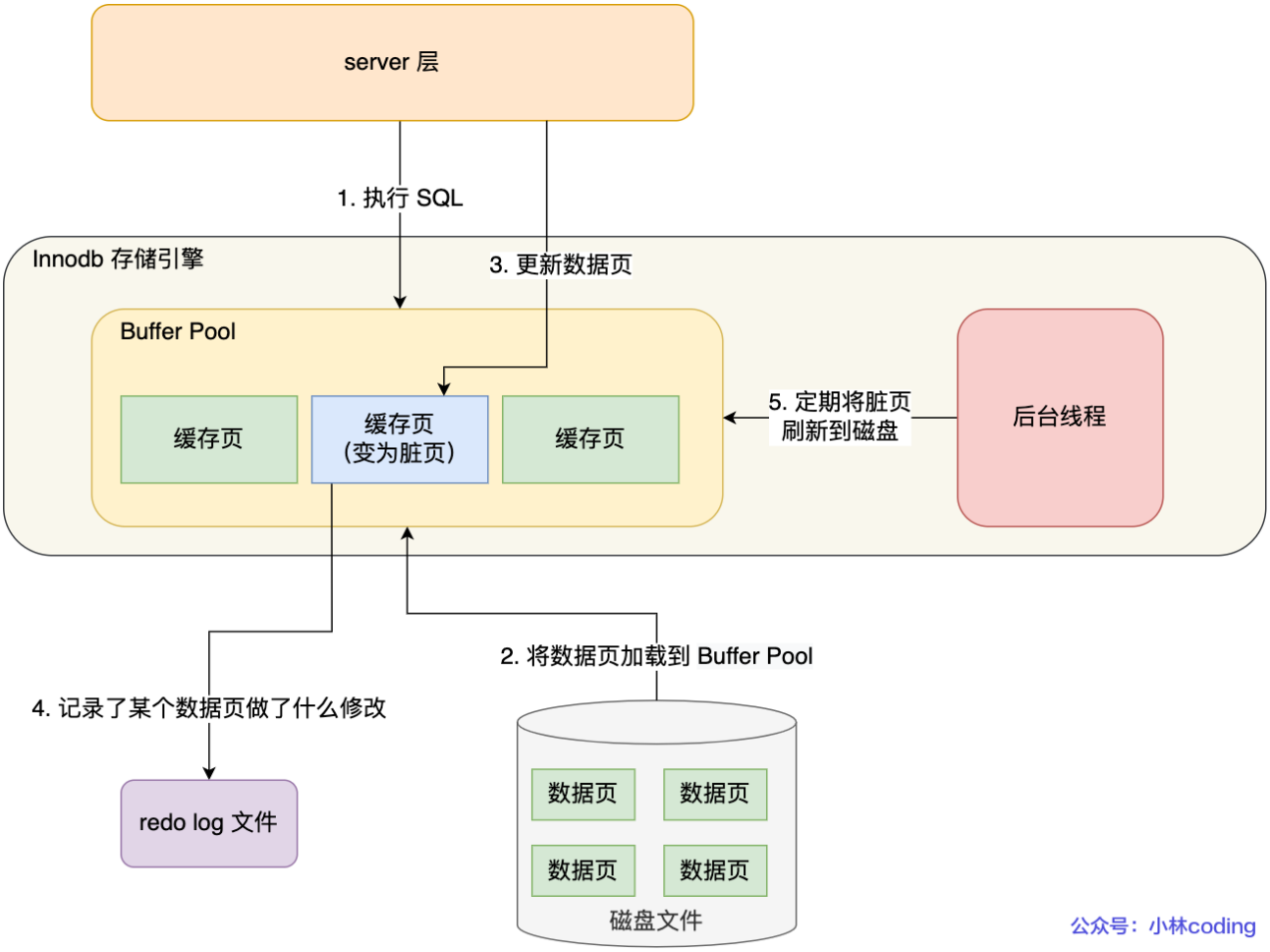
Buffer Pool 是提高了读写效率没错，但是问题来了，Buffer Pool 是基于内存的，而内存总是不可靠，万一断电重启，还没来得及落盘的脏页数据就会丢失。

为了防止断电导致数据丢失的问题，当有一条记录需要更新的时候，InnoDB 引擎就会先更新内存（同时标记为脏页），然后将本次对这个页的修改以 redo log 的形式记录下来，**这个时候更新就算完成了**。

后续，InnoDB 引擎会在适当的时候，由后台线程将缓存在 Buffer Pool 的脏页刷新到磁盘里，这就是 **WAL （Write-Ahead Logging）技术**。

WAL 技术指的是， MySQL 的写操作并不是立刻写到磁盘上，而是先写日志，然后在合适的时间再写到磁盘上。

过程如下图：



redo log 是物理日志，记录了某个数据页做了什么修改，比如**对 XXX 表空间中的 YYY 数据页 ZZZ 偏移量的地方做了 AAA 更新**，每当执行一个事务就会产生这样的一条或者多条物理日志。

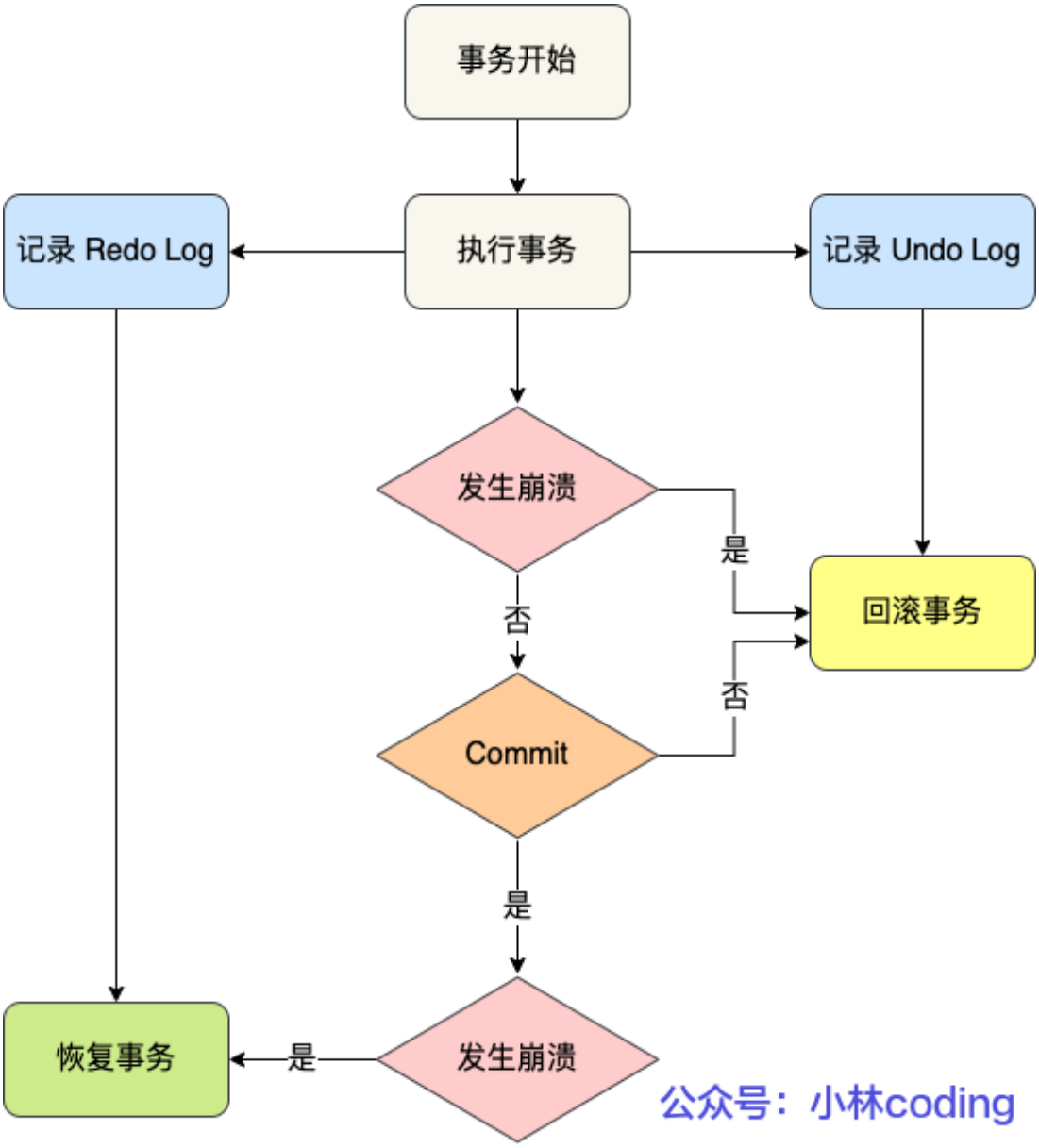
在事务提交时，只要先将 redo log 持久化到磁盘即可，可以不需要等到将缓存在 Buffer Pool 里的脏页数据持久化到磁盘。

当系统崩溃时，虽然脏页数据没有持久化，但是 redo log 已经持久化，接着 MySQL 重启后，可以根据 redo log 的内容，将所有数据恢复到最新的状态。

redo log 和 undo log 这两种日志是属于 InnoDB 存储引擎的日志，它们的区别在于：

- redo log 记录了此次事务「**完成后**」的数据状态，记录的是更新**之后**的值；
- undo log 记录了此次事务「**开始前**」的数据状态，记录的是更新**之前**的值；

事务提交之前发生了崩溃，重启后会通过 undo log 回滚事务，事务提交之后发生了崩溃，重启后会通过 redo log 恢复事务，如下图：



所以有了 redo log，再通过 WAL 技术，InnoDB 就可以保证即使数据库发生异常重启，之前已提交的记录都不会丢失，这个能力称为 crash-safe（崩溃恢复）。可以看出来， **redo log 保证了事务四大特性中的持久性**。

写入 redo log 的方式使用了追加操作， 所以磁盘操作是**顺序写**，而写入数据需要先找到写入位置，然后才写到磁盘，所以磁盘操作是**随机写**。

磁盘的「顺序写 」比「随机写」 高效的多，因此 redo log 写入磁盘的开销更小。

针对「顺序写」为什么比「随机写」更快这个问题，可以比喻为你有一个本子，按照顺序一页一页写肯定比写一个字都要找到对应页写快得多。

可以说这是 WAL 技术的另外一个优点：**MySQL 的写操作从磁盘的「随机写」变成了「顺序写」**，提升语句的执行性能。这是因为 MySQL 的写操作并不是立刻更新到磁盘上，而是先记录在日志上，然后在合适的时间再更新到磁盘上 。

至此， 针对为什么需要 redo log 这个问题我们有两个答案：

- **实现事务的持久性，让 MySQL 有 crash-safe 的能力**，能够保证 MySQL 在任何时间段突然崩溃，重启后之前已提交的记录都不会丢失；
- **将写操作从「随机写」变成了「顺序写」**，提升 MySQL 写入磁盘的性能。

redo log 怎么保证持久性的?

Redo log 是 MySQL 中用于保证持久性的重要机制之一。它通过以下方式来保证持久性：

1. Write-ahead logging（WAL）：在事务提交之前，将事务所做的修改操作记录到 redo log 中，然后再将数据写入磁盘。这样即使在数据写入磁盘之前发生了宕机，系统可以通过 redo log 中的记录来恢复数据。
2. Redo log 的顺序写入：redo log 采用追加写入的方式，将 redo 日志记录追加到文件末尾，而不是随机写入。这样可以减少磁盘的随机 I/O 操作，提高写入性能。
3. Checkpoint 机制：MySQL 会定期将内存中的数据刷新到磁盘，同时将最新的 LSN（Log Sequence Number）记录到磁盘中，这个 LSN 可以确保 redo log 中的操作是按顺序执行的。在恢复数据时，系统会根据 LSN 来确定从哪个位置开始应用 redo log。

能不能只用 binlog 不用 relo log?

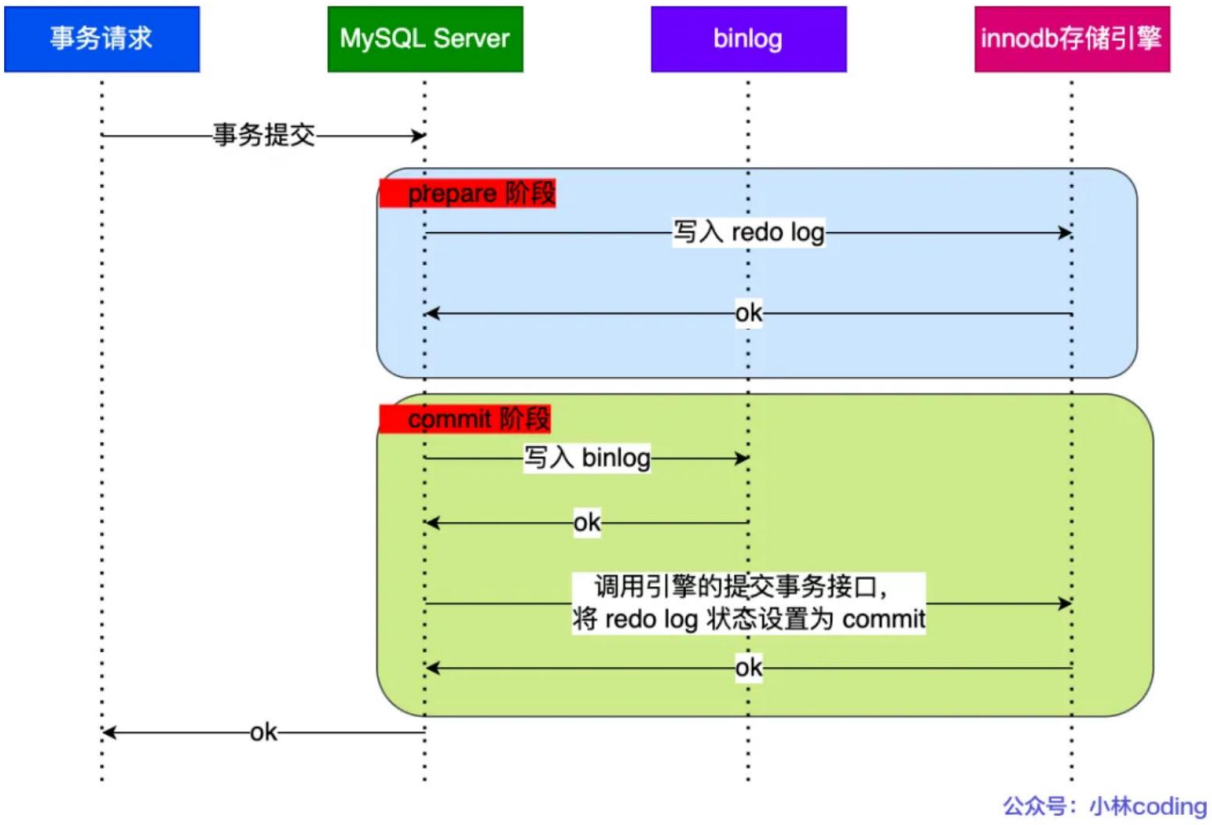
不行，binlog 是 server 层的日志，没办法记录哪些脏页还没有刷盘，redolog 是存储引擎层的日志，可以记录哪些脏页还没有刷盘，这样崩溃恢复的时候，就能恢复那些还没有被刷盘的脏页数据。

binlog 两阶段提交过程是怎么样的?

事务提交后，redo log 和 binlog 都要持久化到磁盘，但是这两个是独立的逻辑，可能出现半成功的状态，这样就造成两份日志之间的逻辑不一致。

在 MySQL 的 InnoDB 存储引擎中，开启 binlog 的情况下，MySQL 会同时维护 binlog 日志与 InnoDB 的 redo log，为了保证这两个日志的一致性，MySQL 使用了**内部 XA 事务**（是的，也有外部 XA 事务，跟本文不太相关，我就不介绍了），内部 XA 事务由 binlog 作为协调者，存储引擎是参与者。

当客户端执行 commit 语句或者在自动提交的情况下，MySQL 内部开启一个 XA 事务，**分两阶段来完成 XA 事务的提交**，如下图：

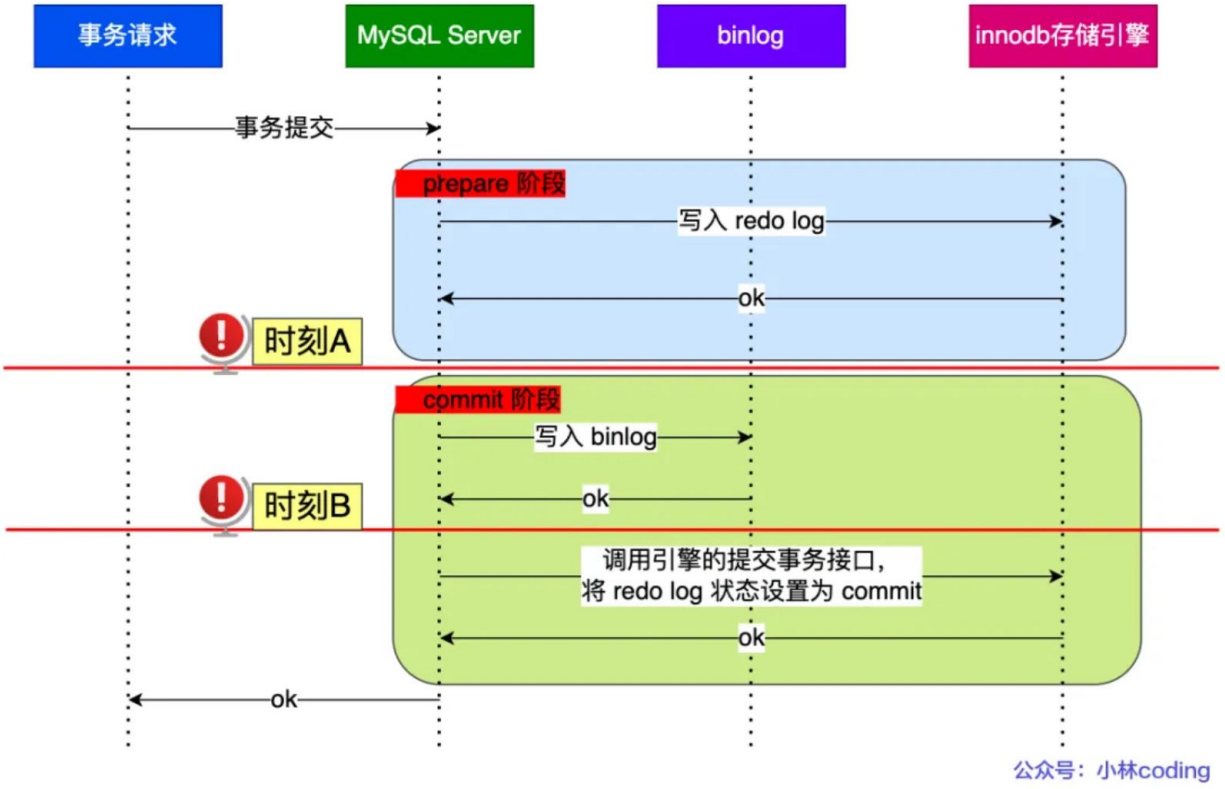


从图中可看出，事务的提交过程有两个阶段，就是**将 redo log 的写入拆成了两个步骤：prepare 和 commit，中间再穿插写入 binlog**，具体如下：

- **prepare 阶段**：将 XID（内部 XA 事务的 ID） 写入到 redo log，同时将 redo log 对应的事务状态设置为 prepare，然后将 redo log 持久化到磁盘（innodb_flush_log_at_trx_commit = 1 的作用）；
- **commit 阶段**：把 XID 写入到 binlog，然后将 binlog 持久化到磁盘（sync_binlog = 1 的作用），接着调用引擎的提交事务接口，将 redo log 状态设置为 commit，此时该状态并不需要持久化到磁盘，只需要 write 到文件系统的 page cache 中就够了，因为只要 binlog

写磁盘成功，就算 redo log 的状态还是 prepare 也没有关系，一样会被认为事务已经执行成功；

我们来看看在两阶段提交的不同时刻，MySQL 异常重启会出现什么现象？下图中有时刻 A 和时刻 B 都有可能发生崩溃：



不管是时刻 A（redo log 已经写入磁盘， binlog 还没写入磁盘），还是时刻 B （redo log 和 binlog 都已经写入磁盘，还没写入 commit 标识）崩溃，**此时的 redo log 都处于 prepare 状态**。

在 MySQL 重启后会按顺序扫描 redo log 文件，碰到处于 prepare 状态的 redo log，就拿着 redo log 中的 XID 去 binlog 查看是否存在此 XID：

- **如果 binlog 中没有当前内部 XA 事务的 XID，说明 redolog 完成刷盘，但是 binlog 还没有刷盘，则回滚事务**。对应时刻 A 崩溃恢复的情况。
- **如果 binlog 中有当前内部 XA 事务的 XID，说明 redolog 和 binlog 都已经完成了刷盘，则提交事务**。对应时刻 B 崩溃恢复的情况。

可以看到，**对于处于 prepare 阶段的 redo log，即可以提交事务，也可以回滚事务，这取决于是否能在 binlog 中查找到与 redo log 相同的 XID**，如果有就提交事务，如果没有就回滚事务。这样就可以保证 redo log 和 binlog 这两份日志的一致性了。

所以说，**两阶段提交是以 binlog 写成功为事务提交成功的标识**，因为 binlog 写成功了，就意味着能在 binlog 中查找到与 redo log 相同的 XID。

update 语句的具体执行过程是怎样的?

具体更新一条记录 `UPDATE t_user SET name = 'xiaolin' WHERE id = 1;` 的流程如下:

1. 执行器负责具体执行，会调用存储引擎的接口，通过主键索引树搜索获取 id = 1 这一行记录：
    - 如果 id=1 这一行所在的数据页本来就在 buffer pool 中，就直接返回给执行器更新；
    - 如果记录不在 buffer pool，将数据页从磁盘读入到 buffer pool，返回记录给执行器。
2. 执行器得到聚簇索引记录后，会看一下更新前的记录和更新后的记录是否一样：
    - 如果一样的话就不进行后续更新流程；
    - 如果不一样的话就把更新前的记录和更新后的记录都当作参数传给 InnoDB 层，让 InnoDB 真正的执行更新记录的操作；
3. 开启事务， InnoDB 层更新记录前，首先要记录相应的 undo log，因为这是更新操作，需要把被更新的列的旧值记下来，也就是要生成一条 undo log，undo log 会写入 Buffer Pool 中的 Undo 页面，不过在内存修改该 Undo 页面后，需要记录对应的 redo log。
4. InnoDB 层开始更新记录，会先更新内存（同时标记为脏页），然后将记录写到 redo log 里面，这个时候更新就算完成了。为了减少磁盘 I/O，不会立即将脏页写入磁盘，后续由后台线程选择一个合适的时机将脏页写入到磁盘。这就是 **WAL 技术**，MySQL 的写操作并不是立刻写到磁盘上，而是先写 redo 日志，然后在合适的时间再将修改的行数据写到磁盘上。
5. 至此，一条记录更新完了。
6. 在一条更新语句执行完成后，然后开始记录该语句对应的 binlog，此时记录的 binlog 会被保存到 binlog cache，并没有刷新到硬盘上的 binlog 文件，在事务提交时才会统一将该事务运行过程中的所有 binlog 刷新到硬盘。
7. 事务提交（为了方便说明，这里不说组提交的过程，只说两阶段提交）：
    - **prepare 阶段**：将 redo log 对应的事务状态设置为 prepare，然后将 redo log 刷新到硬盘；
    - **commit 阶段**：将 binlog 刷新到磁盘，接着调用引擎的提交事务接口，将 redo log 状态设置为 commit（将事务设置为 commit 状态后，刷入到磁盘 redo log 文件）；

8. 至此，一条更新语句执行完成。

MySQL 是如何保障数据不丢失的?

主要是通过 redolog 来实现事务持久性的，事务执行过程，会把对 innodb 存储引擎中数据页修改操作记录到 redolog 里，事务提交的时候，就直接把 redolog 刷入磁盘，即使脏页中途没有刷盘成功， mysql 宕机了，也能通过 redolog 重放，恢复到之前事务修改数据页后的状态，从而保障了数据不丢失。

RedoLog 是在内存里吗?

事务执行过程中，生成的 redolog 会在 redolog buffer 中，也就是在内存中，等事务提交的时候，会把 redolog 写入磁盘。

为什么要写 RedoLog，而不是直接写到 B+树里面?

因为 redolog 写入磁盘是顺序写，而 b+树里数据页写入磁盘是随机写，顺序写的性能会比随机写好，这样可以提升事务提交的效率。

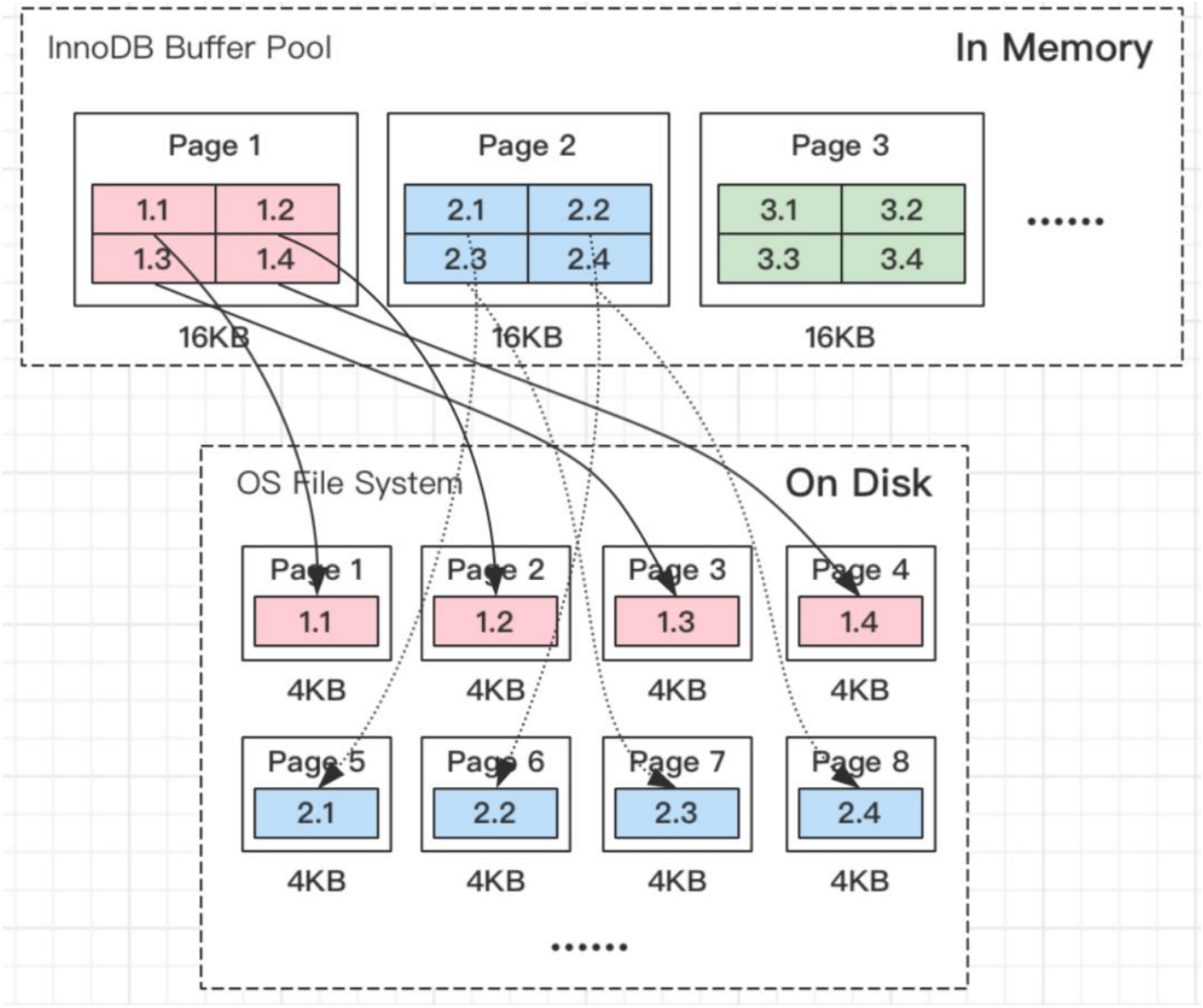
最重要的是 redolog 具备故障恢复的能力，Redo Log 记录的是物理级别的修改，包括页的修改，如插入、更新、删除操作在磁盘上的物理位置和修改内容。例如，当执行一个更新操作时，Redo Log 会记录修改的数据页的地址和更新后的数据，而不是 SQL 语句本身。

在数据页实际更新之前，先将修改操作写入 Redo Log。当数据库重启时，会进行恢复操作。首先，根据 Redo Log 检查哪些事务已经提交但数据页尚未完全写入磁盘。然后，使用 Redo Log 中的记录对这些事务进行重做（Redo）操作，将未完成的数据页修改完成，确保事务的修改生效。

mysql 两次写（double write buffer）了解吗?

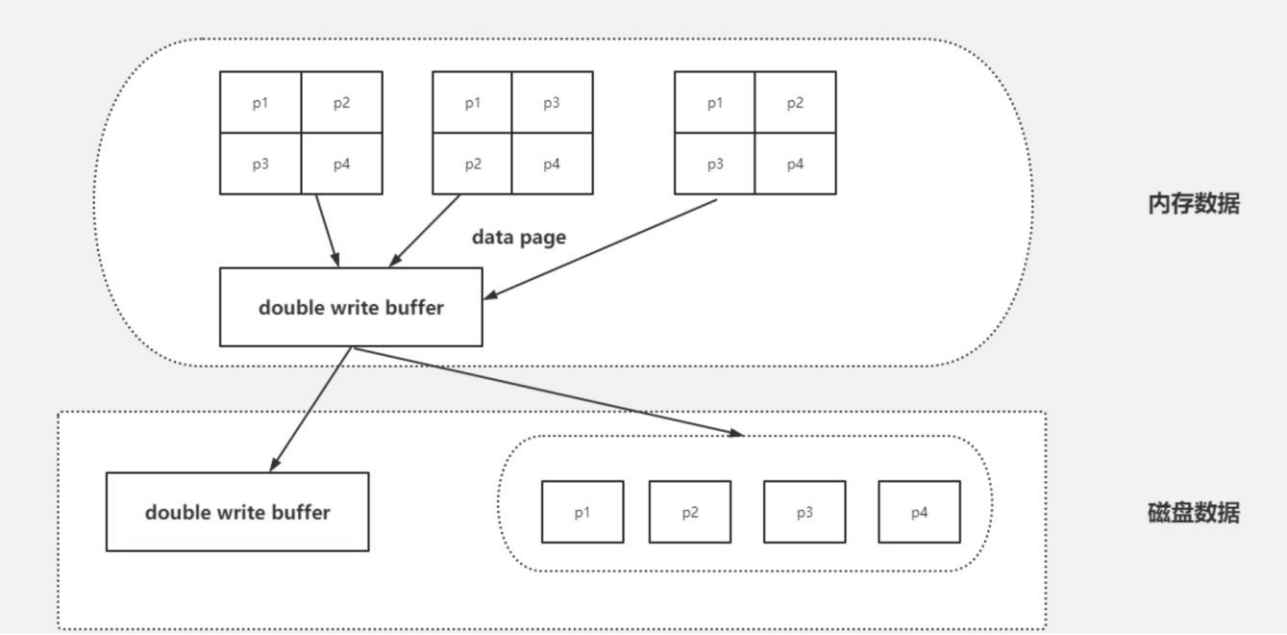
我们常见的服务器一般都是 Linux 操作系统，Linux 文件系统页（OS Page）的大小默认是 4KB。而 MySQL 的页（Page）大小默认是 16KB。

MySQL 程序是跑在 Linux 操作系统上的，需要跟操作系统交互，所以 MySQL 中一页数据刷到磁盘，要写 4 个文件系统里的页。

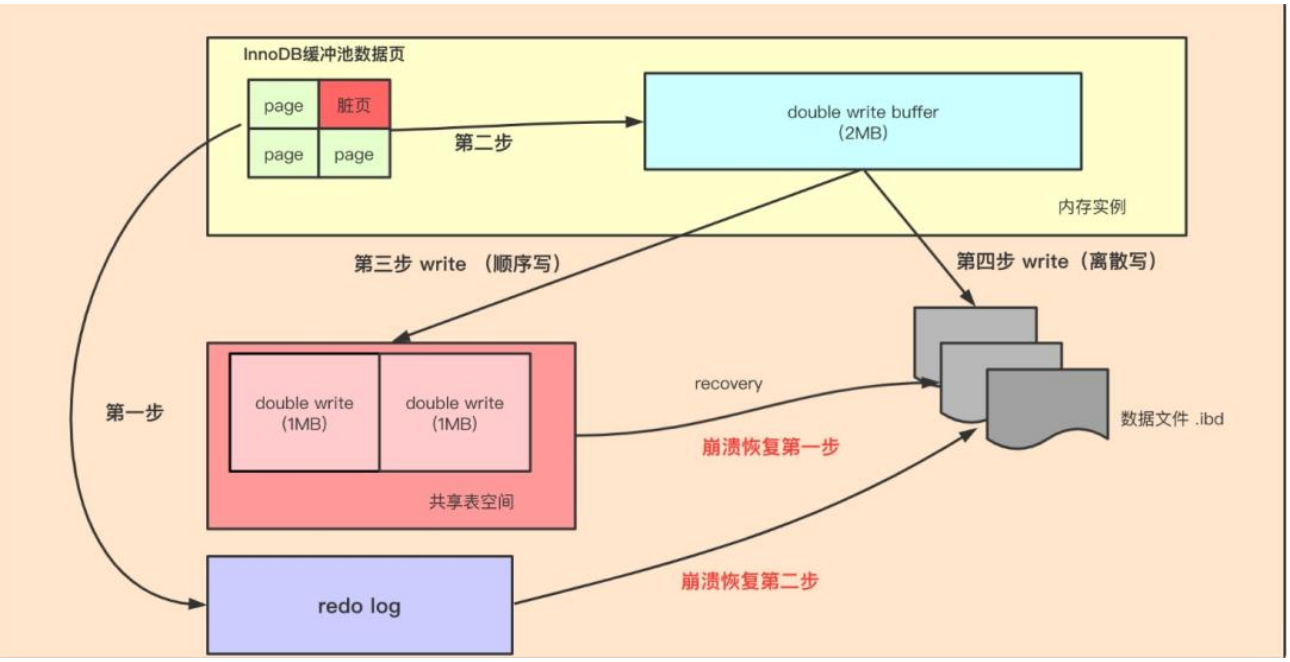


需要注意的是，这个操作并非原子操作，比如我操作系统写到第二个页的时候，Linux 机器断电了，这时候就会出现问题了。造成”页数据损坏“。并且这种”页数据损坏“靠 redo 日志是无法修复的。

Doublewrite Buffer 的出现就是为了解决上面的这种情况，虽然名字带了 Buffer，但实际上 Doublewrite Buffer 是内存+磁盘的结构。



Doublewrite Buffer 作用是，在把页写到数据文件之前，InnoDB 先把它们写到一个叫 doublewrite buffer（双写缓冲区）的共享表空间内，在写 doublewrite buffer 完成后，InnoDB 才会把页写到数据文件的适当的位置。如果在写页的过程中发生意外崩溃，InnoDB 在稍后的恢复过程中在 doublewrite buffer 中找到完好的 page 副本用于恢复，所以本质上是一个最近写回的页面的备份拷贝。



如上图所示，当有页数据要刷盘时：

- 页数据先通过 memcpy 函数拷贝至内存中的 Doublewrite Buffer（大小为约 2MB）中，Doublewrite Buffer 分为两个区域，每次写入一个区域（最多 1MB 的数据）。
- Doublewrite Buffer 的内存里的数据页，会 fsync 刷到 Doublewrite Buffer 的磁盘上，写两次到到共享表空间中(连续存储，顺序写，性能很高)，每次写 1MB；

- 写入完成后，再将脏页刷到数据磁盘存储.ibd 文件上（随机写）；

当 MySQL 出现异常崩溃时，有如下几种情况发生：

- 情况一：步骤 1 前宕机，刷盘未开始，数据在 redo log，后期可以恢复
- 情况二：步骤 1 后，步骤 2 前宕机，因为是在内存中，宕机清空内存，和情况 1 一样
- 情况三：步骤 2 后，步骤 3 前宕机，因为 DWB 的磁盘有完整的数据，可以修复损坏的页数据

由此我们可以得出结论，double write buffer 是针对实际的 buffer 数据页的原子性保证，就是避免 MySQL 异常崩溃时，写的那几个 data page 不会出错，要么都写了，要么什么都没有做。

> 为什么 redolog 无法代替 double write buffer?

redolog 的设计之初，是“账本的作用”，是一种操作日志，用于 MySQL 异常崩溃恢复使用，是 InnoDB 引擎特有的日志，本质上是物理日志，记录的是 “ 在某个数据页上做了什么修改 ” ，但如果数据页本身已经发生了损坏，redolog 来恢复已经损坏的数据块是无效的，数据块的本身已经损坏，再次重做依然是一个坏块。 所以此时需要一个数据块的副本来还原该损坏的数据块，再利用重做日志进行其他数据块的重做操作，这就是 double write buffer 的原因作用。

性能调优

mysql 的 explain 有什么作用?

explain 是查看 sql 的执行计划，主要用来分析 sql 语句的执行过程，比如有没有走索引，有没有外部排序，有没有索引覆盖等等。

如下图，就是一个没有使用索引，并且是一个全表扫描的查询语句。



对于执行计划，参数有：

- possible_keys 字段表示可能用到的索引；
- key 字段表示实际用的索引，如果这一项为 NULL，说明没有使用索引；
- key_len 表示索引的长度；
- rows 表示扫描的数据行数。

- type 表示数据扫描类型，我们需要重点看这个。

type 字段就是描述了找到所需数据时使用的扫描方式是什么，常见扫描类型的**执行效率从低到高的顺序为：**

- All（全表扫描）：在这些情况里，all 是最坏的情况，因为采用了全表扫描的方式。
- index（全索引扫描）：index 和 all 差不多，只不过 index 对索引表进行全扫描，这样做的好处是不再需要对数据进行排序，但是开销依然很大。所以，要尽量避免全表扫描和全索引扫描。
- range（索引范围扫描）：range 表示采用了索引范围扫描，一般在 where 子句中使用 < 、>、in、between 等关键词，只检索给定范围的行，属于范围查找。**从这一级别开始，索引的作用会越来越明显，因此我们需要尽量让 SQL 查询可以使用到 range 这一级别及以上的 type 访问方式**。
- ref（非唯一索引扫描）：ref 类型表示采用了非唯一索引，或者是唯一索引的非唯一性前缀，返回数据返回可能是多条。因为虽然使用了索引，但该索引列的值并不唯一，有重复。这样即使使用索引快速查找到了第一条数据，仍然不能停止，要进行目标值附近的小范围扫描。但它的好处是它并不需要扫全表，因为索引是有序的，即便有重复值，也是在一个非常小的范围内扫描。
- eq_ref（唯一索引扫描）：eq_ref 类型是使用主键或唯一索引时产生的访问方式，通常使用在多表联查中。比如，对两张表进行联查，关联条件是两张表的 user_id 相等，且 user_id 是唯一索引，那么使用 EXPLAIN 进行执行计划查看的时候，type 就会显示 eq_ref。
- const（结果只有一条的主键或唯一索引扫描）：const 类型表示使用了主键或者唯一索引与常量值进行比较，比如 select name from product where id=1。需要说明的是 const 类型和 eq_ref 都使用了主键或唯一索引，不过这两个类型有所区别，**const 是与常量进行比较，查询效率会更快，而 eq_ref 通常用于多表联查中**。

extra 显示的结果，这里说几个重要的参考指标：

- Using filesort ：当查询语句中包含 group by 操作，而且无法利用索引完成排序操作的时候， 这时不得不选择相应的排序算法进行，甚至可能会通过文件排序，效率是很低的，所以要避免这种问题的出现。
- Using temporary：使了用临时表保存中间结果，MySQL 在对查询结果排序时使用临时表，常见于排序 order by 和分组查询 group by。效率低，要避免这种问题的出现。
- Using index：所需数据只需在索引即可全部获得，不须要再到表中取数据，也就是使用了覆盖索引，避免了回表操作，效率不错。

怎么查看是否有走索引?

在 SQL 语句前加 EXPLAIN 关键字，执行后会返回一张执行计划表，重点看几个关键字段就能判断是否走索引。比如要查询用户表中 id 为 100 的用户，先写 EXPLAIN + 目标 SQL：

```
EXPLAIN SELECT * FROM user WHERE id = 100;
```

执行后看结果里的 type 和 key 字段：type 字段如果是 ref、eq_ref 或者 const，说明走了索引；key 字段会显示实际使用的索引名称（比如主键索引 PRIMARY），如果 key 是 NULL，就说明没走索引，走了全表扫描。

再比如如果查询条件是 name 字段，且 name 建了普通索引，执行：

```
EXPLAIN SELECT * FROM user WHERE name = '张三';
```

如果 key 字段显示 name 对应的索引名，type 是 ref，就说明走了索引；如果 key 是 NULL，可能是因为 name 字段的索引选择性太差（比如大部分值都相同），或者查询条件导致索引失效（比如用了函数操作 name 字段，像 `WHERE SUBSTR (name,1,1) = ' 张 '`）。

怎么查看表的索引?

最直接的是 `SHOW INDEX` 命令，语法简单，能清晰列出表的所有索引信息，包括索引名、索引类型、对应字段、是否主键等。比如查看 user 表的索引，执行：

```
SHOW INDEX FROM user;
```

执行后会返回多列结果，关键信息很直观：Key_name 是索引名称（主键索引默认叫 PRIMARY），Column_name 是索引对应的字段，Index_type 是索引类型（比如 BTREE、HASH），Non_unique 字段为 0 表示唯一索引（主键索引也是唯一索引），为 1 表示普通非唯一索引。

如果想更简洁地查看索引名称和对应字段，也可以用 `SHOW KEYS`，它和 `SHOW INDEX` 功能完全一致，只是写法不同：

```
SHOW KEYS FROM user;
```

另外，MySQL 的信息_schema 数据库中存储了所有表的元数据，通过查询 `STATISTICS` 表能获取更详细的索引信息，适合需要筛选或批量查询的场景。比如查询 user 表的索引，执行：

```
SELECT INDEX_NAME, COLUMN_NAME, INDEX_TYPE, NON_UNIQUE
FROM information_schema.STATISTICS
```

```
WHERE TABLE_SCHEMA = '你的数据库名' AND TABLE_NAME = 'user';
```

这里需要替换成实际的数据库名，结果会和 `SHOW INDEX` 一致，但支持更灵活的条件查询，比如只查唯一索引（加 `AND NON_UNIQUE = 0`）。

给你张表，发现查询速度很慢，你有那些解决方案

- **分析查询语句**：使用 EXPLAIN 命令分析 SQL 执行计划，找出慢查询的原因，比如是否使用了全表扫描，是否存在索引未被利用的情况等，并根据相应情况对索引进行适当修改。
- **创建或优化索引**：根据查询条件创建合适的索引，特别是经常用于 WHERE 子句的字段、Orderby 排序的字段、Join 连表查询的字典、 group by 的字段，并且如果查询中经常涉及多个字段，考虑创建联合索引，使用联合索引要符合最左匹配原则，不然会索引失效
- **避免索引失效：**比如不要用左模糊匹配、函数计算、表达式计算等等。
- **查询优化**：避免使用 SELECT *，只查询真正需要的列；使用覆盖索引，即索引包含所有查询的字段；联表查询最好要以小表驱动大表，并且被驱动表的字段要有索引，当然最好通过冗余字段的设计，避免联表查询。
- **分页优化：**针对 limit n,y 深分页的查询优化，可以把 Limit 查询转换成某个位置的查询：select * from tb_sku where id>20000 limit 10，该方案适用于主键自增的表，
- **优化数据库表**：如果单表的数据超过了千万级别，考虑是否需要将大表拆分为小表，减轻单个表的查询压力。也可以将字段多的表分解成多个表，有些字段使用频率高，有些低，数据量大时，会由于使用频率低的存在而变慢，可以考虑分开。
- **使用缓存技术**：引入缓存层，如 Redis，存储热点数据和频繁查询的结果，但是要考虑缓存一致性的问题，对于读请求会选择旁路缓存策略，对于写请求会选择先更新 db，再删除缓存的策略。

如果 Explain 用到的索引不正确的话，有什么办法干预吗?

可以使用 force index，强制走索引。

比如：

```
EXPLAIN SELECT
    productName, buyPrice
FROM
    products
FORCE INDEX (idx_buyprice)
```

```
WHERE
    buyPrice BETWEEN 10 AND 80
ORDER BY buyPrice;
```

输出：

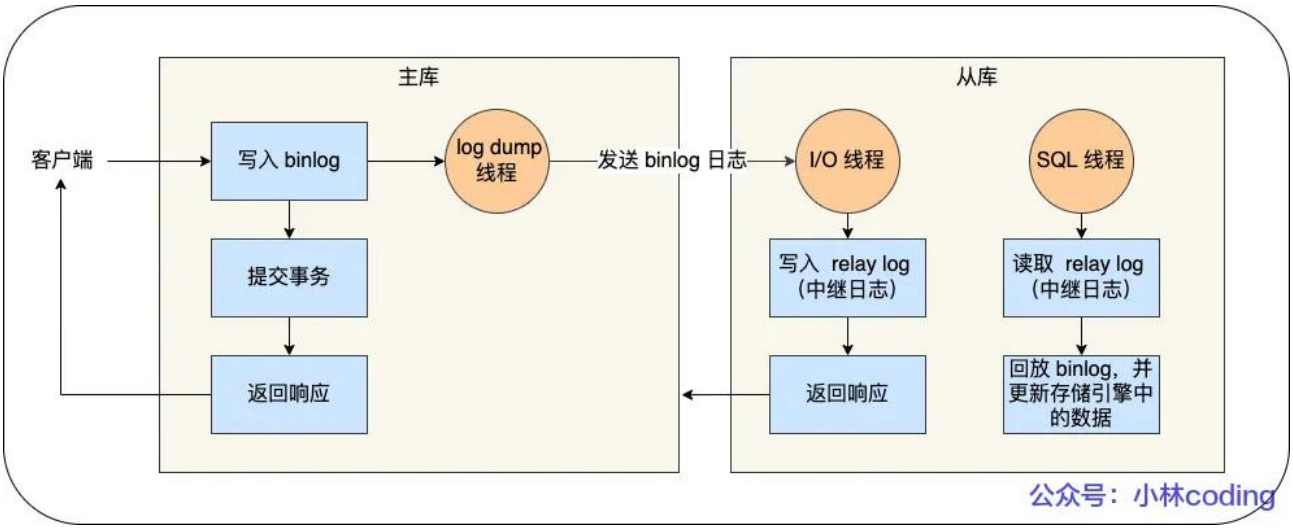
```
+----+-------------+----------+------------+-------+---------------+---------------+---------+------+------+----------+-----------------------+
| id | select_type | table    | partitions | type  | possible_keys | key           | key_len | ref  | rows | filtered | Extra                 |
+----+-------------+----------+------------+-------+---------------+---------------+---------+------+------+----------+-----------------------+
|  1 | SIMPLE      | products | NULL       | range | idx_buyprice  | idx_buyprice | 5       | NULL |   94 |   100.00 | Using index condition |
+----+-------------+----------+------------+-------+---------------+---------------+---------+------+------+----------+-----------------------+
1 row in set, 1 warning (0.10 sec)
```

架构

MySQL 主从复制了解吗

MySQL 的主从复制依赖于 binlog ，也就是记录 MySQL 上的所有变化并以二进制形式保存在磁盘上。复制的过程就是将 binlog 中的数据从主库传输到从库上。

这个过程一般是**异步**的，也就是主库上执行事务操作的线程不会等待复制 binlog 的线程同步完成。



MySQL 集群的主从复制过程梳理成 3 个阶段：

- **写入 Binlog**：主库写 binlog 日志，提交事务，并更新本地存储数据。
- **同步 Binlog**：把 binlog 复制到所有从库上，每个从库把 binlog 写到暂存日志中。
- **回放 Binlog**：回放 binlog，并更新存储引擎中的数据。

具体详细过程如下：

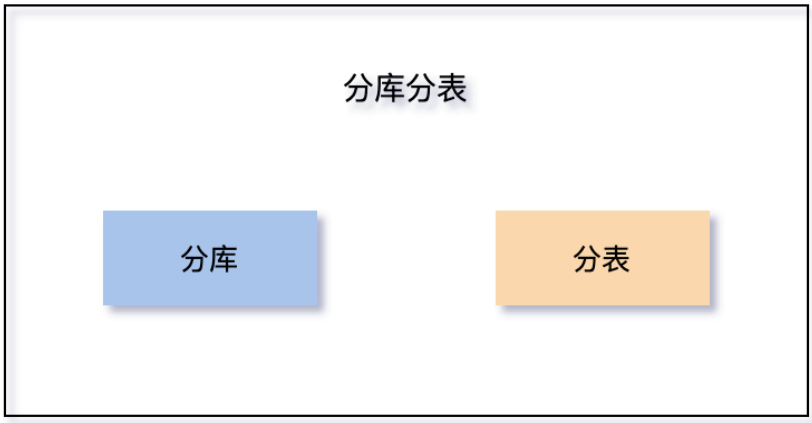
- MySQL 主库在收到客户端提交事务的请求之后，会先写入 binlog，再提交事务，更新存储引擎中的数据，事务提交完成后，返回给客户端“操作成功”的响应。
- 从库会创建一个专门的 I/O 线程，连接主库的 log dump 线程，来接收主库的 binlog 日志，再把 binlog 信息写入 relay log 的中继日志里，再返回给主库“复制成功”的响应。
- 从库会创建一个用于回放 binlog 的线程，去读 relay log 中继日志，然后回放 binlog 更新存储引擎中的数据，最终实现主从的数据一致性。

在完成主从复制之后，你就可以在写数据时只写主库，在读数据时只读从库，这样即使写请求会锁表或者锁记录，也不会影响读请求的执行。

主从延迟都有什么处理方法?

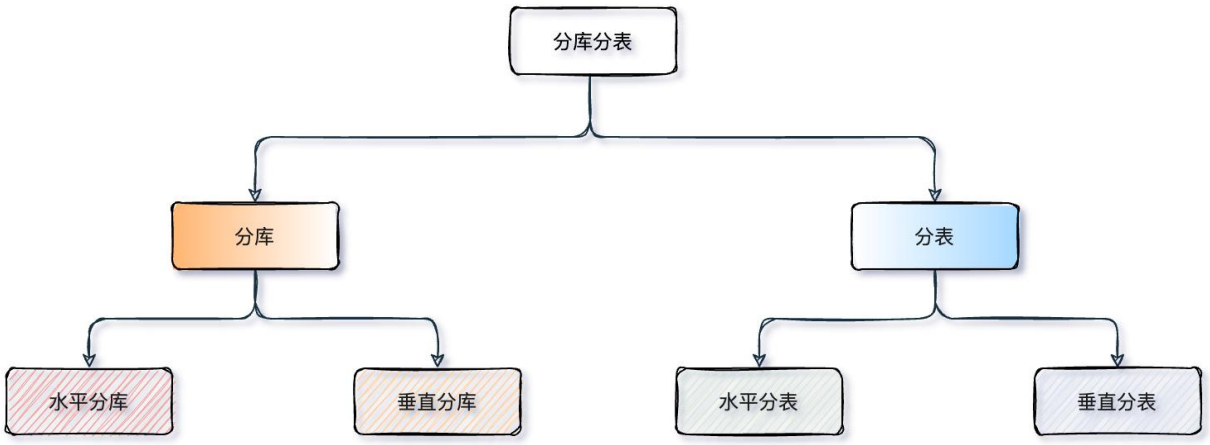
强制走主库方案：对于大事务或资源密集型操作，直接在主库上执行，避免从库的额外延迟。

分表和分库是什么？有什么区别?



- **分库**是一种水平扩展数据库的技术，将数据根据一定规则划分到多个独立的数据库中。每个数据库只负责存储部分数据，实现了数据的拆分和分布式存储。分库主要是为了解决并发连接过多，单机 mysql 扛不住的问题。
- **分表**指的是将单个数据库中的表拆分成多个表，每个表只负责存储一部分数据。这种数据的垂直划分能够提高查询效率，减轻单个表的压力。分表主要是为了解决单表数据量太大，导致查询性能下降的问题。

分库与分表可以从：垂直（纵向）和 水平（横向）两种纬度进行拆分。下边我们以经典的订单业务举例，看看如何拆分。



- **垂直分库**：一般来说按照业务和功能的维度进行拆分，将不同业务数据分别放到不同的数据库中，核心理念 专库专用。按业务类型对数据分离，剥离为多个数据库，像订单、支付、会员、积分相关等表放在对应的订单库、支付库、会员库、积分库。垂直分库把一个库的压力分摊到多个库，提升了一些数据库性能，但并没有解决由于单表数据量过大导致的性能问题，所以就需要配合后边的分表来解决。
- **垂直分表**：针对业务上字段比较多的大表进行的，一般是把业务宽表中比较独立的字段，或者不常用的字段拆分到单独的数据表中，是一种大表拆小表的模式。数据库它是以行为单位将数据加载到内存中，这样拆分以后核心表大多是访问频率较高的字段，而且字段长度也都较短，因而可以加载更多数据到内存中，减少磁盘 IO，增加索引查询的命中率，进一步提升数据库性能。
- **水平分库**：是把同一个表按一定规则拆分到不同的数据库中，每个库可以位于不同的服务器上，以此实现水平扩展，是一种常见的提升数据库性能的方式。这种方案往往能解决单库存储量及性能瓶颈问题，但由于同一个表被分配在不同的数据库中，数据的访问需要额外的路由工作，因此系统的复杂度也被提升了。
- **水平分表**：是在**同一个数据库内**，把一张大数据量的表按一定规则，切分成多个结构完全相同表，而每个表只存原表的一部分数据。水平分表尽管拆分了表，但子表都还是在同一个数据库实例中，只是解决了单一表数据量过大的问题，并没有将拆分后的表分散到不同的机器上，还在竞争同一个物理机的 CPU、内存、网络 IO 等。要想进一步提升性能，就需要将拆分后的表分散到不同的数据库中，达到分布式的效果。
